

Manipulator mit sechs Freiheitsgraden

1

Erzeugt von Doxygen 1.14.0

Kapitel 1

Verzeichnis der Namensbereiche

1.1 Liste aller Namensbereiche

Liste aller Namensbereiche mit Kurzbeschreibung:

Logger		??
Utils		??

Kapitel 2

Klassen-Verzeichnis

2.1 Auflistung der Klassen

Hier folgt die Aufzählung aller Klassen, Strukturen, Varianten und Schnittstellen mit einer Kurzbeschreibung:

Actuator	??
Command	??
Commands	??
DhParam	??
DhParameters	??
Joint	??
Kinematic	??
Matrix4x4	??
PosData	??
Position	??
PositionFactory	??
Posture	??
RobFrame	??
sPosition	??
sPosture	??

Kapitel 3

Datei-Verzeichnis

3.1 Auflistung der Dateien

Hier folgt die Aufzählung aller Dateien mit einer Kurzbeschreibung:

Actuator.cpp	??
Actuator.hpp	??
Commands.cpp	??
Commands.hpp	??
Joint.cpp	??
Joint.hpp	??
Kinematic.cpp	??
Kinematic.hpp	??
Logger.cpp	??
Logger.hpp	??
main.cpp	??
pins.h	??
PositionFactory.cpp	??
PositionFactory.hpp	??
RobFrameWork.cpp	??
RobFrameWork.hpp	??
Types.hpp	??
Utilities.cpp	??
Utilities.hpp	??

Kapitel 4

Dokumentation der Namensbereiche

4.1 Logger-Namensbereichsreferenz

Funktionen

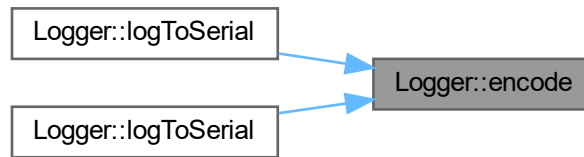
- void `logInit` (long baudRate)
- int `Incoming` ()
- void `encode` (String data)
- void `log` (String text="")
- void `logInt` (int digit=-1)
- void `logft` (float dec=-1)
- void `log` (String text="", String titel="")
- void `log` (int digit=-1, String titel="")
- void `logft` (float dec=-1.0, String titel="")
- void `log` (`Posture` pt)
- void `logIn` (`Posture` pt)
- void `logIn` (`Posture` pt, String title)
- String `ptEncode` (float val)
- void `logToSerial` (`Posture` pt)
- void `logToSerial` (String input)
- void `log` (`Position` ps)
- void `logIn` (`Position` ps)
- void `log` (`Position` ps, String title="Location and Orientation of Endeffector")
- void `logIn` (`Position` ps, String title)
- void `log` (`Command` cmd)
- void `log` (`PosData` pdata, `PosState` state)

4.1.1 Dokumentation der Funktionen

4.1.1.1 `encode()`

```
void Logger::encode (  
    String data)
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



4.1.1.2 Incoming()

```
int Logger::Incoming ()
```

4.1.1.3 log() [1/8]

```
void Logger::log (  
    Command cmd)
```

4.1.1.4 log() [2/8]

```
void Logger::log (  
    int digit = -1,  
    String titel = "")
```

4.1.1.5 log() [3/8]

```
void Logger::log (  
    PosData pdata,  
    PosState state)
```

4.1.1.6 log() [4/8]

```
void Logger::log (  
    Position ps)
```

4.1.1.7 log() [5/8]

```
void Logger::log (  
    Position ps,  
    String title = "Location and Orientation of Endeffector")
```

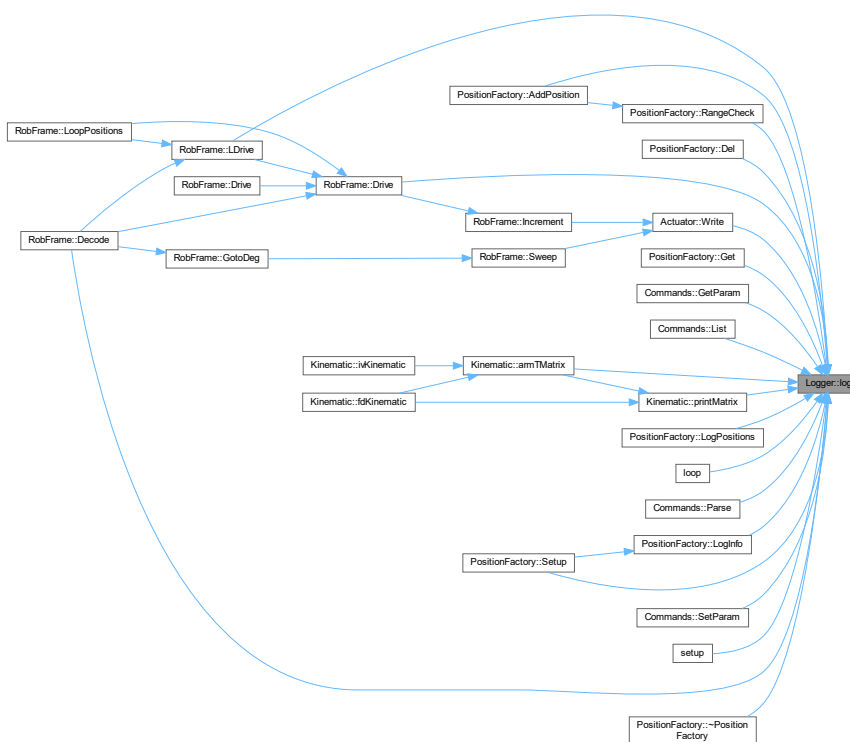
4.1.1.8 log() [6/8]

```
void Logger::log (
    Posture pt)
```

4.1.1.9 log() [7/8]

```
void Logger::log (
    String text = "")
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



4.1.1.10 log() [8/8]

```
void Logger::log (
    String text = "",
    String titel = "")
```

4.1.1.11 logft() [1/2]

```
void Logger::logft (
    float dec = -1)
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



4.1.1.12 logft() [2/2]

```
void Logger::logft (
    float dec = -1.0,
    String titel = "")
```

4.1.1.13 logInit()

```
void Logger::logInit (
    long baudRate)
```

4.1.1.14 logInt()

```
void Logger::logInt (
    int digit = -1)
```

4.1.1.15 logln() [1/4]

```
void Logger::logln (
    Position ps)
```

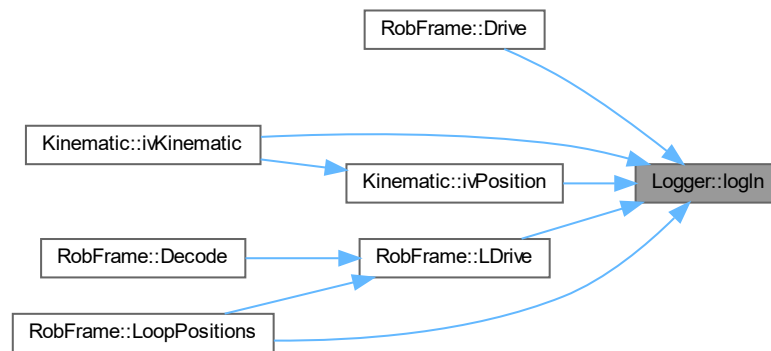
4.1.1.16 logln() [2/4]

```
void Logger::logln (
    Position ps,
    String title)
```

4.1.1.17 logIn() [3/4]

```
void Logger::logIn (  
    Posture pt)
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

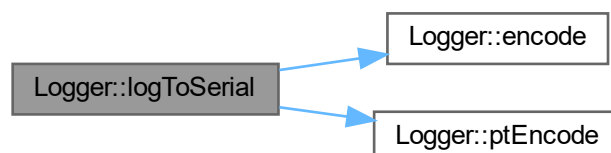
**4.1.1.18 logIn() [4/4]**

```
void Logger::logIn (  
    Posture pt,  
    String title)
```

4.1.1.19 logToSerial() [1/2]

```
void Logger::logToSerial (  
    Posture pt)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



4.1.1.20 logToSerial() [2/2]

```
void Logger::logToSerial (  
    String input)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



4.1.1.21 ptEncode()

```
String Logger::ptEncode (  
    float val)
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



4.2 Utils-Namensbereichsreferenz

Funktionen

- `word SetBit16` (`word` var, `int8` index)
- `word GetBit16` (`word` var, `int8` index)
- `word ResetBit16` (`word` var, `int8` index)

4.2.1 Dokumentation der Funktionen

4.2.1.1 GetBit16()

```
word Utils::GetBit16 (
    word var,
    int8 index)
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



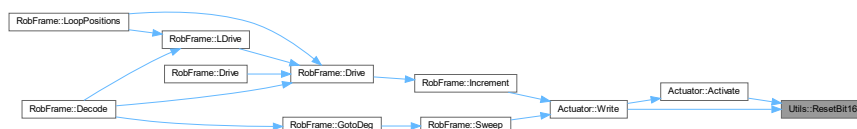
4.2.1.2 ResetBit16()

```
word Utils::ResetBit16 (
    word var,
    int8 index)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



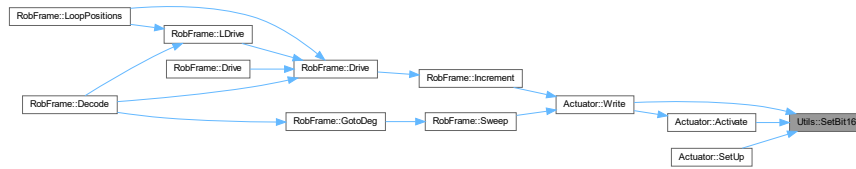
Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



4.2.1.3 SetBit16()

```
word Utils::SetBit16 (  
    word var,  
    int8 index)
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



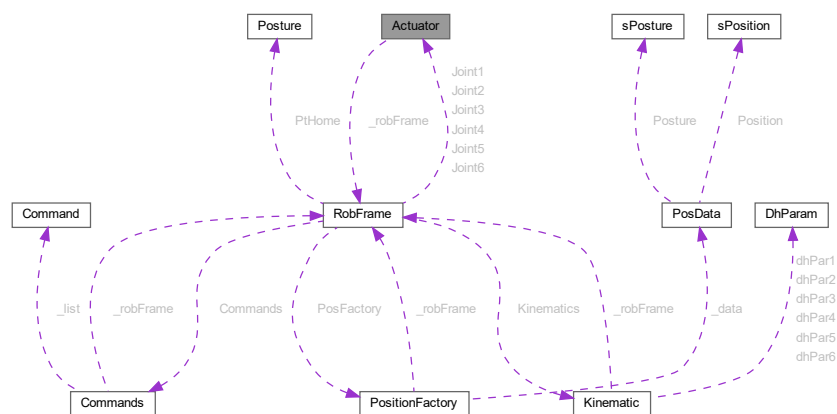
Kapitel 5

Klassen-Dokumentation

5.1 Actuator Klassenreferenz

```
#include <Actuator.hpp>
```

Zusammengehörigkeiten von Actuator:



Öffentliche Typen

- enum `State` {
 `Enabled` , `Stopped` , `InMotion` , `Referenced` ,
 `InPosition` , `Aborted` }

Actuator Betriebszustand.

Öffentliche Methoden

- [Actuator](#) ()
Actor Klassen-Objekt.
- void [Init](#) ([RobFrame](#) *robFrame)
Initialisierung Zeiger-Zuweisung.
- [int16 Id](#) ()
Aktor ID.
- [int16 SetUp](#) (float gearRatio=1, [int16](#) accuracy=5, [int16](#) max=90, [int16](#) min=-90)
Hardwareeinrichten.
- [int16 Attach](#) ([int16](#) dirPin, [int16](#) stepPin, [int16](#) enaPin)
Hardware-Ausgänge Zuweisen Anbindung der Hardware-Ausgänge.
- [int32 Activate](#) ([int8](#) ena, [uint16](#) tof=5000)
Aktoransteuerung.
- [int32 Reference](#) ([int32](#) value=0)
IstPositionswert Referenzieren.
- [int32 Write](#) ([int32](#) target)
Aktuator Sollwert Schreiben.
- [int32 Read](#) ()
lese Istposition
- [int32 Delta](#) ([int32](#) target)
Soll-Ist Differenze bilden.

Öffentliche Attribute

- [int16 Accuracy](#)
Genauigkeit.

Private Methoden

- [int32 calcSteps](#) ([int32](#) target)
Winkelwerte in Grad zu Schritte konvertieren.
- [int32 calcPosition](#) ([int32](#) actSteps)
Schritte zu Winkel.

Private Attribute

- [RobFrame](#) * [_robFrame](#)
Zeiger zu [RobFrame](#).
- [BasicStepperDriver](#) * [_stepper](#)
Zeiger zur Stepper Objekt.
- [uint8 _motorSteps](#) = 200
- [uint8 _microSteps](#) = 16
- [int8 _dir](#) = 1
- [int16 _id](#) = 0
- [int16 _offset](#)
- [int16 _min](#) = -90
- [int16 _max](#) = 90
- [int16 _gearRatio](#) = 1
- [int32 _previousSteps](#) = 0
- [int32 _actualSteps](#) = 0
- [int32 _targetSteps](#) = 0
- [uint16 _timeOfDelay](#) = 0
- [float _rpm](#) = 10
- [int8 _enabled](#) = -1
- [word _state](#) = 0

5.1.1 Dokumentation der Aufzählungstypen

5.1.1.1 State

```
enum Actuator::State
```

Aktuator Betriebszustand.

Aufzählungswerte

	Enabled
	Stopped
	InMotion
	Referenced
	InPosition
	Aborted

5.1.2 Beschreibung der Konstruktoren und Destruktoren

5.1.2.1 Actuator()

```
Actuator::Actuator ()
```

Actor Klassen-Objekt.

Instanzierung des Aktuators bzw. Steppermotor Klassenobject.

5.1.3 Dokumentation der Elementfunktionen

5.1.3.1 Activate()

```
int32 Actuator::Activate (  
    int8 ena,  
    uint16 tof = 5000)
```

Aktoransteuerung.

Asynchrones Aktoransteuerung Einschalten bzw. Freigeben.

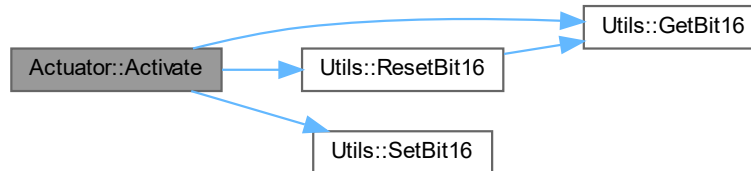
Parameter

	ena	Aktivieren
	tof	Ausschaltverzögerung

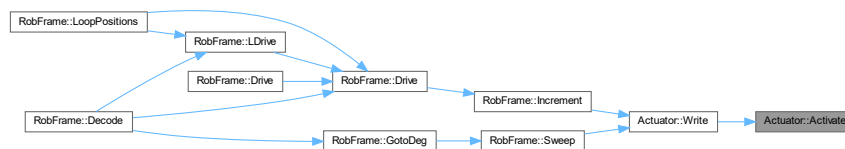
Rückgabe

ON wenn activ oder OFF wenn nicht Active

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**5.1.3.2 Attach()**

```

int16 Actuator::Attach (
    int16 dirPin,
    int16 stepPin,
    int16 enaPin)

```

Hardware-Ausgänge Zuweisen Anbinddung der Hardware-Ausgänge.

Parameter

<code>dirPin</code>	Richtungs Pin id.
<code>stepPin</code>	Stepper Pin id.
<code>enaPin</code>	Aktivierungs Pin id.

Rückgabe

Zeiger-Id zur BasicSteperDriver objekt

Anbindung der Hardwareausgänge am Klassen-Ojekt durch die Initialisierung der "BasicStepperDriver" Klasse

5.1.3.3 calcPosition()

```

int32 Actuator::calcPosition (
    int32 actSteps) [private]

```

Schritte zu Winkel.

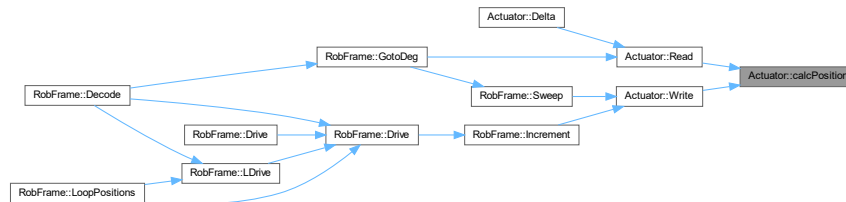
Parameter

	<i>Schritte</i>
--	-----------------

Rückgabe

Winkel

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**5.1.3.4 calcSteps()**

```
int32 Actuator::calcSteps (
    int32 target) [private]
```

Winkelwerte in Grad zu Schritte konvertieren.

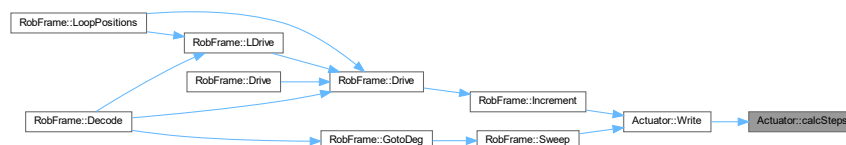
Parameter

	<i>target</i>	Winkel
--	---------------	--------

Rückgabe

void

Private Functions / Prozeduren Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**5.1.3.5 Delta()**

```
int32 Actuator::Delta (
    int32 target)
```

Soll-Ist Differenz bilden.

Soll-Ist Differenz bilden in Grad.

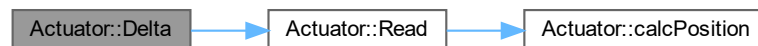
Parameter

<code>target</code>	Zielposition
---------------------	--------------

Rückgabe

Winkel differenze

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:

**5.1.3.6 Id()**

```
int16 Actuator::Id ()
```

Aktor ID.

Eindeutiges ID auslesen.

Public Funktions / Prozeduren

5.1.3.7 Init()

```
void Actuator::Init (  
    RobFrame * robFrame)
```

Initialisierung Zeiger-Zuweisung.

RoboterFrameWork-Objekt Zeigerzuweisung.

Parameter

<code>robFrame</code>	Zeiger zu RobFrame
-----------------------	------------------------------------

Rückgabe

void

5.1.3.8 Read()

```
int32 Actuator::Read ()
```

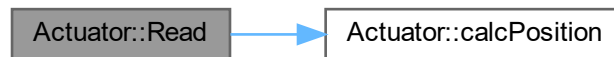
lese Istposition

Ist-Winkelwert von in Grad auslesen.

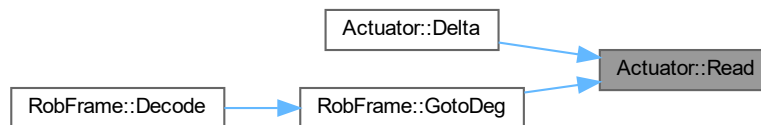
Rückgabe

Istwert in Grad

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.1.3.9 Refernce()

```
int32 Actuator::Refernce (
    int32 value = 0)
```

IstPositionswert Referenzieren.

Referenzierung der gespeicherten Ist-Position.

Parameter

<code>value</code>	Zurücksetzungswert
--------------------	--------------------

Rückgabe

neuer Istposition

5.1.3.10 SetUp()

```
int16 Actuator::SetUp (
    float gearRatio = 1,
    int16 accuracy = 5,
    int16 max = 90,
    int16 min = -90)
```

Hardwareeinrichten.

Hardware eigenschaften einrichten.

Parameter

	<i>gearRatio</i>	Getriebeübersetzung
	<i>accuracy</i>	Genauigkeit
	<i>max</i>	Max-Winkelstellung
	<i>min</i>	Min-Winkelstellung

Rückgabe

OK wenn Erfolgreich

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.1.3.11 Write()

```
int32 Actuator::Write (
    int32 target)
```

Aktuator Sollwert Schreiben.

Sollpositon Synchrone Schreiben.

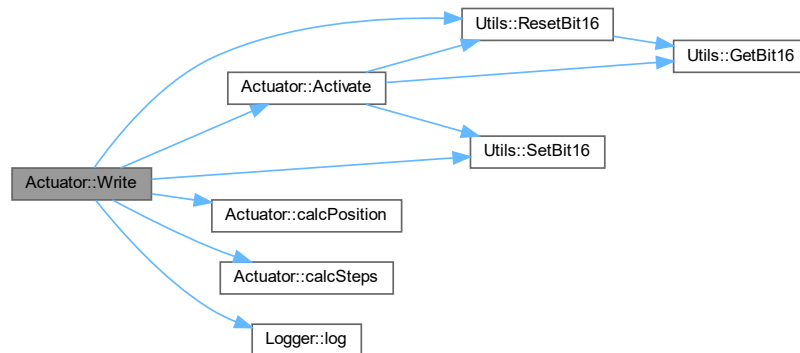
Parameter

	<i>target</i>	Zielvorgabe in Grad
--	---------------	---------------------

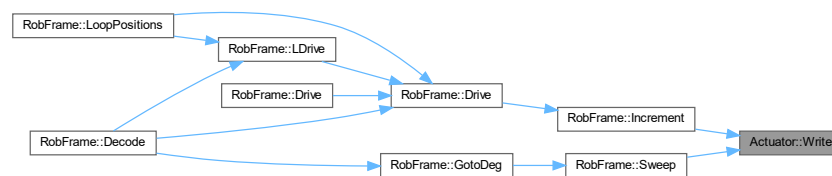
Rückgabe

Anzahl gefahrene Schritte.

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.1.4 Dokumentation der Datenelemente

5.1.4.1 `_actualSteps`

```
int32 Actuator::_actualSteps = 0 [private]
```

5.1.4.2 `_dir`

```
int8 Actuator::_dir = 1 [private]
```

5.1.4.3 `_enabled`

```
int8 Actuator::_enabled = -1 [private]
```

5.1.4.4 `_gearRatio`

```
int16 Actuator::_gearRatio = 1 [private]
```

5.1.4.5 `_id`

```
int16 Actuator::_id = 0 [private]
```

5.1.4.6 `_max`

```
int16 Actuator::_max = 90 [private]
```

5.1.4.7 `_microSteps`

```
uint8 Actuator::_microSteps = 16 [private]
```

5.1.4.8 `_min`

```
int16 Actuator::_min = -90 [private]
```

5.1.4.9 `_motorSteps`

```
uint8 Actuator::_motorSteps = 200 [private]
```

5.1.4.10 `_offset`

```
int16 Actuator::_offset [private]
```

5.1.4.11 `_previousSteps`

```
int32 Actuator::_previousSteps = 0 [private]
```

5.1.4.12 `_robFrame`

```
RobFrame* Actuator::_robFrame [private]
```

Zeiger zu [RobFrame](#).

5.1.4.13 `_rpm`

```
float Actuator::_rpm = 10 [private]
```

5.1.4.14 `_state`

```
word Actuator::_state = 0 [private]
```

5.1.4.15 `_stepper`

```
BasicStepperDriver* Actuator::_stepper [private]
```

Zeiger zur Stepper Objekt.

5.1.4.16 `_targetSteps`

```
int32 Actuator::_targetSteps = 0 [private]
```

5.1.4.17 `_timeOfDelay`

```
uint16 Actuator::_timeOfDelay = 0 [private]
```

5.1.4.18 **Accuracy**

```
int16 Actuator::Accuracy
```

Genauigkeit.

Die Dokumentation für diese Klasse wurde erzeugt aufgrund der Dateien:

- [Actuator.hpp](#)
- [Actuator.cpp](#)

5.2 Command Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- int `Id`
- String `Name`
- int `Value`
- String `Content`

5.2.1 Dokumentation der Datenelemente

5.2.1.1 **Content**

```
String Command::Content
```

5.2.1.2 Id

```
int Command::Id
```

5.2.1.3 Name

```
String Command::Name
```

5.2.1.4 Value

```
int Command::Value
```

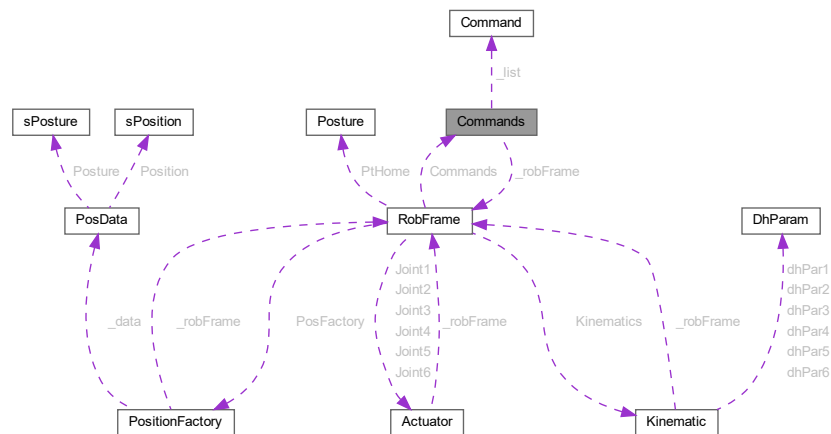
Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.3 Commands Klassenreferenz

```
#include <Commands.hpp>
```

Zusammengehörigkeiten von Commands:



Öffentliche Typen

- enum **Tags** {
[World](#) , [Jt1](#) , [Jt2](#) , [Jt3](#) ,
[Jt4](#) , [Jt5](#) , [Jt6](#) , [Jt7](#) ,
[Here](#) , [Joints](#) , [GoHome](#) , [Drive](#) ,
[JDrive](#) , [LDrive](#) , [Speed](#) , [Accuracy](#) ,
[Debug](#) , [Voltage](#) , [Position](#) , [Posture](#) ,
[ListPositions](#) , [ListAllPositions](#) , [JLoop](#) ,
[LLoop](#) , [Rotate](#) , [RotStep](#) , [Help](#) }

Öffentliche Methoden

- `Commands ()`
- `Command & operator[] (int index)`
- `Command Parse (String &input)`
- `Command Get (Tags tag)`
- `void Init (RobFrame *robFrame)`
- `int GetParam (int cIndex)`
- `void SetParam (int cIndex, float val)`
- `void Delay ()`
- `void List ()`

Private Attribute

- `int _range = 0`
- `RobFrame * _robFrame`
- `Command _list [COMMAND_LEN]`

5.3.1 Dokumentation der Aufzählungstypen

5.3.1.1 Tags

```
enum Commands::Tags
```

Aufzählungswerte

	World
	Jt1
	Jt2
	Jt3
	Jt4
	Jt5
	Jt6
	Jt7
	Here
	Joints
	GoHome
	Drive
	JDrive
	LDrive
	Speed
	Accuracy
	Debug
	Voltage
	Position
	Posture
	ListPositions
	ListPostures
	ListAllPositions
	JLoop
	LLoop
	Rotate
	RotStep
	Help

5.3.2 Beschreibung der Konstruktoren und Destruktoren

5.3.2.1 Commands()

```
Commands::Commands ()
```

5.3.3 Dokumentation der Elementfunktionen

5.3.3.1 Delay()

```
void Commands::Delay ()
```

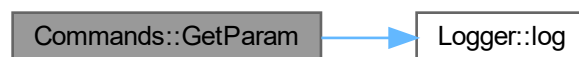
5.3.3.2 Get()

```
Command Commands::Get (  
    Commands::Tags tag)
```

5.3.3.3 GetParam()

```
int Commands::GetParam (  
    int cIndex)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.3.3.4 Init()

```
void Commands::Init (  
    RobFrame * robFrame)
```

5.3.3.5 List()

```
void Commands::List ()
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.3.3.6 operator[]()

```
Command & Commands::operator[] (
    int index)
```

5.3.3.7 Parse()

```
Command Commands::Parse (
    String & input)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.3.3.8 SetParam()

```
void Commands::SetParam (
    int cIndex,
    float val)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.3.4 Dokumentation der Datenelemente

5.3.4.1 `_list`

```
Command Commands::_list[COMMAND_LEN] [private]
```

5.3.4.2 `_range`

```
int Commands::_range = 0 [private]
```

5.3.4.3 `_robFrame`

```
RobFrame* Commands::_robFrame [private]
```

Die Dokumentation für diese Klasse wurde erzeugt aufgrund der Dateien:

- [Commands.hpp](#)
- [Commands.cpp](#)

5.4 DhParam Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- float [alpha](#)
- float [beta](#)
- float [theta](#)
- float [link](#)
- float [disp](#)

5.4.1 Dokumentation der Datenelemente

5.4.1.1 `alpha`

```
float DhParam::alpha
```

5.4.1.2 `beta`

```
float DhParam::beta
```

5.4.1.3 disp

```
float DhParam::disp
```

5.4.1.4 link

```
float DhParam::link
```

5.4.1.5 theta

```
float DhParam::theta
```

Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.5 DhParameters Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- float [alpha](#) [6] = { 90.000, 0.000, 0.000, 0.000, 0.000, 0.000}
- float [beta](#) [6] = {-90.000, 0.000, 90.000, -90.000, 90.000, 0.000}
- float [theta](#) [6] = { 0.000, 0.000, 0.000, 0.000, 0.000, 0.000}
- float [link](#) [6] = { 0.000, 258.300, 281.516, 0.000, 74.710, 0.000}
- float [disp](#) [6] = { 0.000, 0.000, 0.000, 0.000, 0.000, 0.000}

5.5.1 Dokumentation der Datenelemente

5.5.1.1 alpha

```
float DhParameters::alpha[6] = { 90.000, 0.000, 0.000, 0.000, 0.000, 0.000}
```

5.5.1.2 beta

```
float DhParameters::beta[6] = {-90.000, 0.000, 90.000, -90.000, 90.000, 0.000}
```

5.5.1.3 disp

```
float DhParameters::disp[6] = { 0.000, 0.000, 0.000, 0.000, 0.000, 0.000}
```

5.5.1.4 link

```
float DhParameters::link[6] = { 0.000, 258.300, 281.516, 0.000, 74.710, 0.000 }
```

5.5.1.5 theta

```
float DhParameters::theta[6] = { 0.000, 0.000, 0.000, 0.000, 0.000, 0.000 }
```

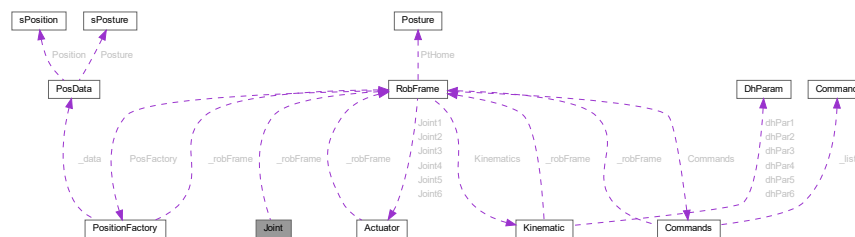
Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.6 Joint Klassenreferenz

```
#include <Joint.hpp>
```

Zusammengehörigkeiten von Joint:



Öffentliche Methoden

- [Joint](#) ()
- void [Init](#) ([RobFrame](#) *robFrame)
- void [write](#) (int degree)
- void [attach](#) (int pin)
- void [attach](#) (int pin, int [min](#), int [max](#))
- void [attach](#) (int pin, int [min](#), int [max](#), int [offset](#))
- void [limit](#) (int [min](#), int [max](#))
- int [read](#) ()
- int [read2](#) ()
- int [readMicroseconds](#) ()
- int [delta](#) (int dest)

Öffentliche Attribute

- int [id](#) = 0
- int [offset](#) = 0
- int [min](#) = 1
- int [max](#) = 180
- Servo [servo](#)

Private Attribute

- `int _offset`
- `RobFrame * _robFrame`

5.6.1 Beschreibung der Konstruktoren und Destruktoren

5.6.1.1 Joint()

```
Joint::Joint ()
```

5.6.2 Dokumentation der Elementfunktionen

5.6.2.1 attach() [1/3]

```
void Joint::attach (  
    int pin)
```

5.6.2.2 attach() [2/3]

```
void Joint::attach (  
    int pin,  
    int min,  
    int max)
```

5.6.2.3 attach() [3/3]

```
void Joint::attach (  
    int pin,  
    int min,  
    int max,  
    int offset)
```

5.6.2.4 delta()

```
int Joint::delta (  
    int dest)
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.6.2.5 Init()

```
void Joint::Init (  
    RobFrame * robFrame)
```

5.6.2.6 limit()

```
void Joint::limit (  
    int min,  
    int max)
```

5.6.2.7 read()

```
int Joint::read ()
```

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.6.2.8 read2()

```
int Joint::read2 ()
```

5.6.2.9 readMicroseconds()

```
int Joint::readMicroseconds ()
```

5.6.2.10 write()

```
void Joint::write (  
    int degree)
```

5.6.3 Dokumentation der Datenelemente

5.6.3.1 _offset

```
int Joint::_offset [private]
```

5.6.3.2 `_robFrame`

```
RobFrame* Joint::_robFrame [private]
```

5.6.3.3 `id`

```
int Joint::id = 0
```

5.6.3.4 `max`

```
int Joint::max = 180
```

5.6.3.5 `min`

```
int Joint::min = 1
```

5.6.3.6 `offset`

```
int Joint::offset = 0
```

5.6.3.7 `servo`

```
Servo Joint::servo
```

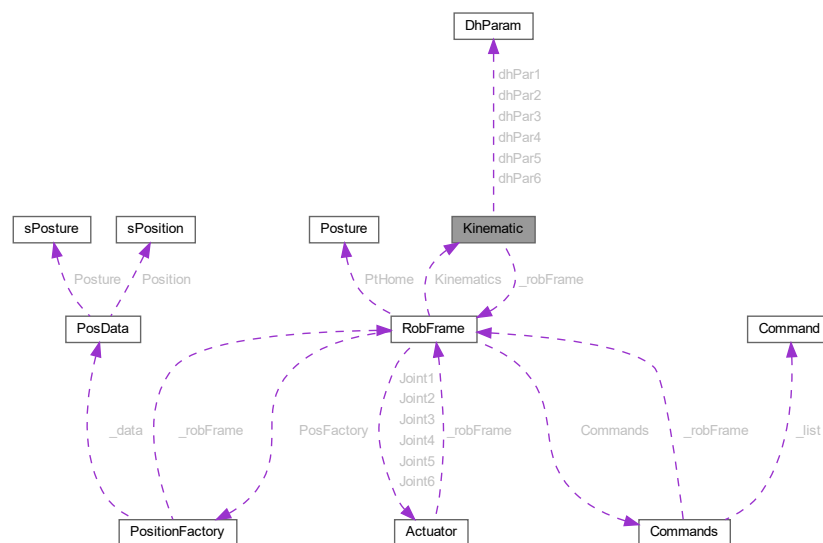
Die Dokumentation für diese Klasse wurde erzeugt aufgrund der Dateien:

- [Joint.hpp](#)
- [Joint.cpp](#)

5.7 Kinematic Klassenreferenz

```
#include <Kinematic.hpp>
```

Zusammengehörigkeiten von Kinematic:



Öffentliche Methoden

- [Kinematic](#) ()
Kinematic Klassen-Objekt.
- void [Init](#) ([RobFrame](#) *robFrame)
Initialisierung Zeiger zuweisung.
- void [printMatrix](#) (tmm::Matrix< 4, 4 > mat, String heading)
Matrix drucken.
- [Position](#) [fdKinematic](#) ([Posture](#) posture)
Vorwaertskinematik berechnung.
- [Posture](#) [ivKinematic](#) ([Position](#) position)
Inverskinematik berechnung.
- [Posture](#) [ivPosition](#) ([Position](#) position)
Positionsgebenden Winkeln Berechnen.
- [Posture](#) [ivOrientation](#) ([Posture](#) posture, [Position](#) ps)
Orientierungsgebenden Winkeln Berechnen.

Öffentliche Attribute

- [DhParam](#) [dhPar1](#) = {}
D-H Parameter Gelenk 1.
- [DhParam](#) [dhPar2](#) = {}
D-H Parameter Gelenk 2.
- [DhParam](#) [dhPar3](#) = {}
D-H Parameter Gelenk 3.
- [DhParam](#) [dhPar4](#) = {}
D-H Parameter Gelenk 4.
- [DhParam](#) [dhPar5](#) = {}
D-H Parameter Gelenk 5.
- [DhParam](#) [dhPar6](#) = {}
D-H Parameter Gelenk 6.
- tmm::Matrix< 4, 4 > [tMat01](#) = {}
Achsen-Transformations-Matrix 1.
- tmm::Matrix< 4, 4 > [tMat12](#) = {}
Achsen-Transformations-Matrix 2.
- tmm::Matrix< 4, 4 > [tMat23](#) = {}
Achsen-Transformations-Matrix 3.
- tmm::Matrix< 4, 4 > [tMat34](#) = {}
Achsen-Transformations-Matrix 4.
- tmm::Matrix< 4, 4 > [tMat45](#) = {}
Achsen-Transformations-Matrix 5.
- tmm::Matrix< 4, 4 > [tMat56](#) = {}
Achsen-Transformations-Matrix 6.
- tmm::Matrix< 4, 4 > [tMat02](#) = {}
Verkettet Transformations-Matrix 02.
- tmm::Matrix< 4, 4 > [tMat03](#) = {}
Verkettet Transformations-Matrix 03.
- tmm::Matrix< 4, 4 > [tMat04](#) = {}
Verkettet Transformations-Matrix 04.
- tmm::Matrix< 4, 4 > [tMat05](#) = {}
Verkettet Transformations-Matrix 05.
- tmm::Matrix< 4, 4 > [tMat06](#) = {}
Verkettet Transformations-Matrix 06.

Private Methoden

- `tmm::Matrix< 4, 4 > jointTMatrix (DhParam dhPar)`
Homogenen Transformations-Matrix des gelenks.
- `tmm::Matrix< 4, 4 > armTMatrix (Posture posture)`
Homogenen Transformations-Matrix des Manipulators.
- `tmm::Matrix< 3, 3 > read3x3 (tmm::Matrix< 4, 4 > tMat)`
Rotations-Matrix auslesen.

Private Attribute

- `RobFrame * _robFrame`
Zeiger zu RobFrame.

5.7.1 Beschreibung der Konstruktoren und Destruktoren

5.7.1.1 Kinematic()

`Kinematic::Kinematic ()`

`Kinematic` Klassen-Objekt.

Klassen-Objekt instanzieren und Denavite-Hartenberg Parameter zuweisen.

5.7.2 Dokumentation der Elementfunktionen

5.7.2.1 armTMatrix()

```
tmm::Matrix< 4, 4 > Kinematic::armTMatrix (
    Posture p) [private]
```

Homogenen Transformations-Matrix des Manipulators.

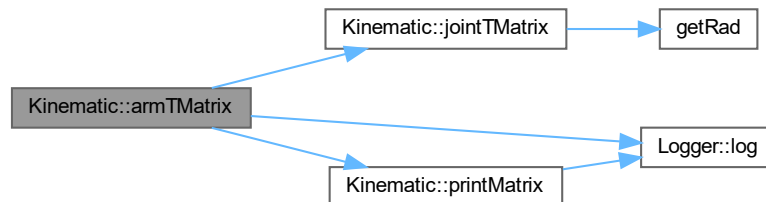
Parameter

	<i>Manipulator-Stellung</i>	Theta 1 bis 6
--	-----------------------------	---------------

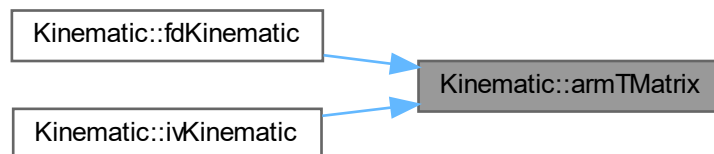
Rückgabe

Transformations-Matrix

Homogenen Transformations-Matrix des Manipulators aufstellen und wiedergeben mit bezug zu Gelenkwinkeln theta 1 bis 6 Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.7.2.2 fdKinematic()

```
Position Kinematic::fdKinematic (
    Posture pt)
```

Vorwaertskinematik berechnung.

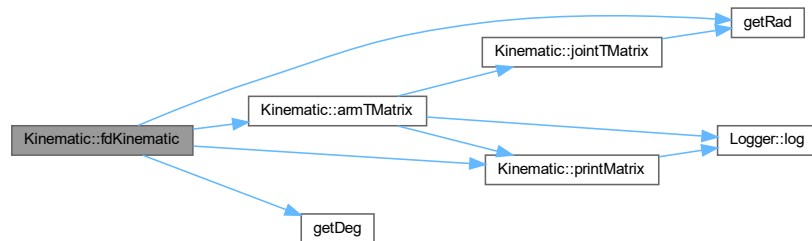
Parameter

<code>posture</code>	Manipulator Winkelstellung
----------------------	----------------------------

Rückgabe

Lage ([Position](#) und Orientierung)

Vorwärtskinematik berechnung anhand der Denavite-Hartenberg Konvention Hier ist ein Graph, der zeigt, was diese Funktion aufruft:

**5.7.2.3 Init()**

```
void Kinematic::Init (
    RobFrame * robFrame)
```

Initialisierung Zeiger zuweisung.

Initialisierung Roboter-Framework Zeiger zuweisen.

Parameter

<code>robFrame</code>	Zeiger zu RobFrame
-----------------------	------------------------------------

Rückgabe

void

5.7.2.4 ivKinematic()

```
Posture Kinematic::ivKinematic (
    Position p6)
```

Inverskinematik berechnung.

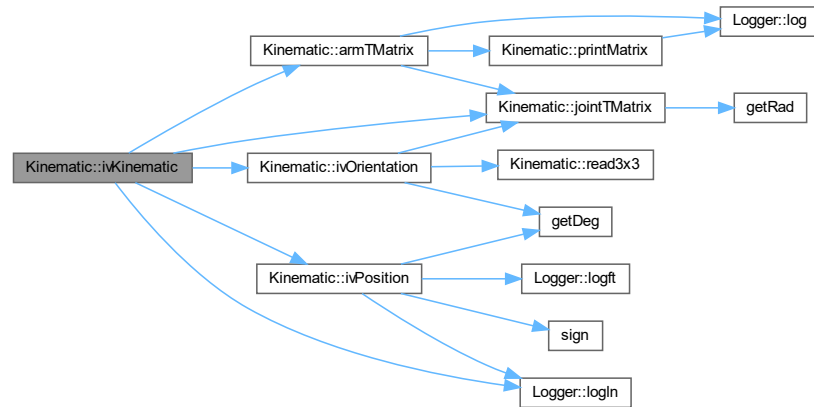
Parameter

<code>position</code>	Manipulator Position
-----------------------	--------------------------------------

Rückgabe

Manipulator Winkelstellung

Inverskinematik Berechnung anhand der Denavite-Hartenberge Konvention Hier ist ein Graph, der zeigt, was diese Funktion aufruft:

**5.7.2.5 ivOrientation()**

```

Posture Kinematic::ivOrientation (
    Posture pt,
    Position ps)
  
```

Orientierungsgebenden Winkeln Berechnen.

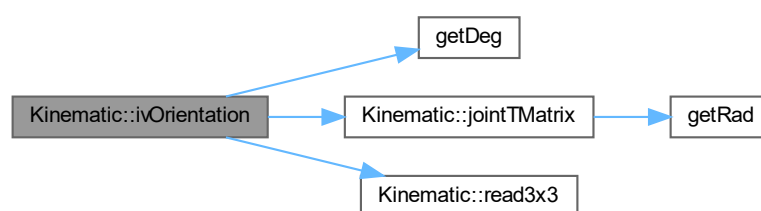
Parameter

<i>posture</i>	Winkelstellung
<i>Ps</i>	Position

Rückgabe

Winkelstellung

Ermittlung der Orientierung/Gelenkstellung d.h. th4, th5 und th6 Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.7.2.6 ivPosition()

```
Posture Kinematic::ivPosition (  
    Position ps)
```

Positionsgebenden Winkeln Berechnen.

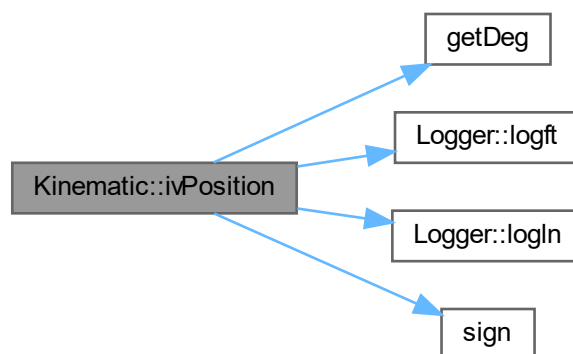
Parameter

<i>position</i>	lage Position und Orientierung
-----------------	---------------------------------------

Rückgabe

Winkelwerte 1...3

Berechnung der Positionsgebenden Winkelstellungen d.h. th1, th2 und th3 Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.7.2.7 jointTMatrix()

```

tmm::Matrix< 4, 4 > Kinematic::jointTMatrix (
    DhParam dhPar) [private]
  
```

Homogenen Transformations-Matrix des gelenks.

Parameter

<code>dhPar</code>	Denavite-Hartenberge Parameter
--------------------	--------------------------------

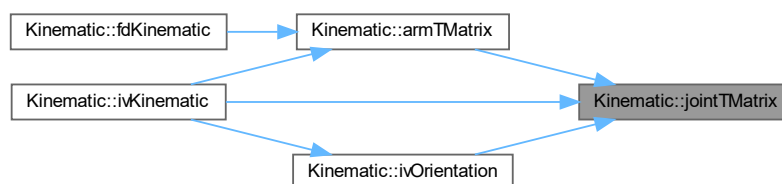
Rückgabe

Transformations-Matrix

Homogenen Transformations-Matrix des Gelenks aufstellen und wiedergeben mit bezug auf die Denavite-Hartenberg Parameter Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.7.2.8 printMatrix()

```
void Kinematic::printMatrix (
    tmm::Matrix< 4, 4 > mat,
    String heading)
```

Matrix drucken.

4x4 Matrix in Seriellen Monitor ausgeben

Parameter

	<i>mat</i>	4x4 Matrix
	<i>heading</i>	Matrix Tag

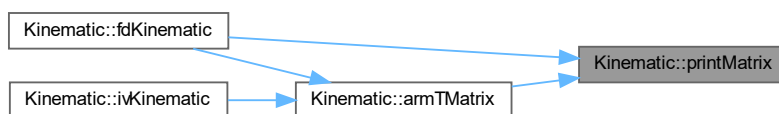
Rückgabe

void

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.7.2.9 read3x3()

```
tmm::Matrix< 3, 3 > Kinematic::read3x3 (
    tmm::Matrix< 4, 4 > tMat) [private]
```

Rotations-Matrix auslesen.

Parameter

<code>tMat</code>	Transformations-Matrix
-------------------	------------------------

3x3 Rotations-Matrix aus gegebenen 4x4 Transformations-Matrix auslesen. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.7.3 Dokumentation der Datenelemente

5.7.3.1 `_robFrame`

```
RobFrame* Kinematic::_robFrame [private]
```

Zeiger zu [RobFrame](#).

5.7.3.2 `dhPar1`

```
DhParam Kinematic::dhPar1 = {}
```

D-H Parameter Gelenk 1.

5.7.3.3 `dhPar2`

```
DhParam Kinematic::dhPar2 = {}
```

D-H Parameter Gelenk 2.

5.7.3.4 `dhPar3`

```
DhParam Kinematic::dhPar3 = {}
```

D-H Parameter Gelenk 3.

5.7.3.5 `dhPar4`

```
DhParam Kinematic::dhPar4 = {}
```

D-H Parameter Gelenk 4.

5.7.3.6 dhPar5

```
DhParam Kinematic::dhPar5 = {}
```

D-H Parameter Gelenk 5.

5.7.3.7 dhPar6

```
DhParam Kinematic::dhPar6 = {}
```

D-H Parameter Gelenk 6.

5.7.3.8 tMat01

```
tm::Matrix<4,4> Kinematic::tMat01 = {}
```

Achsen-Transformations-Matrix 1.

5.7.3.9 tMat02

```
tm::Matrix<4,4> Kinematic::tMat02 = {}
```

Verkettet Transformations-Matrix 02.

5.7.3.10 tMat03

```
tm::Matrix<4,4> Kinematic::tMat03 = {}
```

Verkettet Transformations-Matrix 03.

5.7.3.11 tMat04

```
tm::Matrix<4,4> Kinematic::tMat04 = {}
```

Verkettet Transformations-Matrix 04.

5.7.3.12 tMat05

```
tm::Matrix<4,4> Kinematic::tMat05 = {}
```

Verkettet Transformations-Matrix 05.

5.7.3.13 tMat06

```
tm::Matrix<4,4> Kinematic::tMat06 = {}
```

Verkettet Transformations-Matrix 06.

5.7.3.14 tMat12

```
tm::Matrix<4,4> Kinematic::tMat12 = {}
```

Achsen-Transformations-Matrix 2.

5.7.3.15 tMat23

```
tm::Matrix<4,4> Kinematic::tMat23 = {}
```

Achsen-Transformations-Matrix 3.

5.7.3.16 tMat34

```
tm::Matrix<4,4> Kinematic::tMat34 = {}
```

Achsen-Transformations-Matrix 4.

5.7.3.17 tMat45

```
tm::Matrix<4,4> Kinematic::tMat45 = {}
```

Achsen-Transformations-Matrix 5.

5.7.3.18 tMat56

```
tm::Matrix<4,4> Kinematic::tMat56 = {}
```

Achsen-Transformations-Matrix 6.

Die Dokumentation für diese Klasse wurde erzeugt aufgrund der Dateien:

- [Kinematic.hpp](#)
- [Kinematic.cpp](#)

5.8 Matrix4x4 Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- float [m11](#) [4]
- float [m12](#) [4]
- float [m13](#) [4]
- float [m14](#) [4]

5.8.1 Dokumentation der Datenelemente

5.8.1.1 m11

```
float Matrix4x4::m11[4]
```

5.8.1.2 m12

```
float Matrix4x4::m12[4]
```

5.8.1.3 m13

```
float Matrix4x4::m13[4]
```

5.8.1.4 m14

```
float Matrix4x4::m14[4]
```

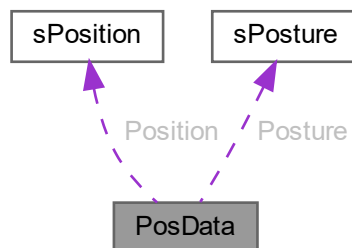
Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.9 PosData Strukturreferenz

```
#include <Types.hpp>
```

Zusammengehörigkeiten von PosData:



Öffentliche Attribute

- `char` [Identifier](#) [[IDENTIFIER_LEN](#)]
- [sPosition](#) `Position`
- [sPosture](#) `Posture`

5.9.1 Dokumentation der Datenelemente

5.9.1.1 Identifier

```
char PosData::Identifier[IDENTIFIER_LEN]
```

5.9.1.2 Position

```
sPosition PosData::Position
```

5.9.1.3 Posture

```
sPosture PosData::Posture
```

Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.10 Position Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- float [X](#)
- float [Y](#)
- float [Z](#)
- float [A](#)
- float [B](#)
- float [C](#)

5.10.1 Dokumentation der Datenelemente

5.10.1.1 A

```
float Position::A
```

5.10.1.2 B

```
float Position::B
```

5.10.1.3 C

```
float Position::C
```

5.10.1.4 X

```
float Position::X
```

5.10.1.5 Y

```
float Position::Y
```

5.10.1.6 Z

```
float Position::Z
```

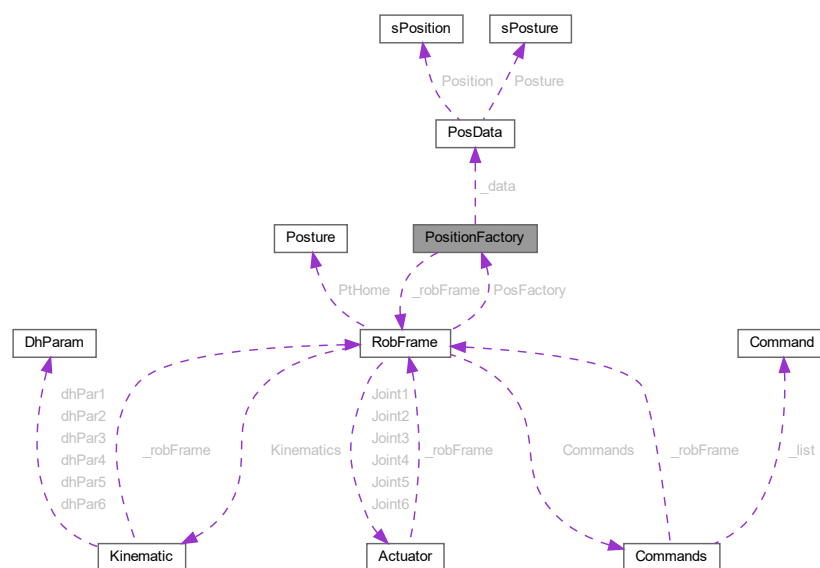
Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.11 PositionFactory Klassenreferenz

```
#include <PositionFactory.hpp>
```

Zusammengehörigkeiten von PositionFactory:



Öffentliche Methoden

- [PositionFactory](#) ()
Klassen-Objekt erstellen.
- [~PositionFactory](#) ()
Datenarray Vernichten.
- void [Init](#) ([RobFrame](#) *robFrame)
RobFrameWork Zeigerzuweisung und Speicherplatz allozieren.
- void [Setup](#) ()
Initialisierung.
- [Posture DecodePosture](#) (String input, String filter=",")
String to [Position](#).
- [Position DecodePosition](#) (String input, String filter=",")
String to [Position](#).
- int & [AddPosition](#) (String content, [PosState](#) state, char filter=' ')
[Position](#), Winkelstellung hinzufügen.
- int & [Count](#) ()
Datensatzanzahl.
- void [Get](#) (String identifier)
nicht Implementiert
- void [Del](#) (String identifier)
nicht Implementiert
- void [LogInfo](#) ()
Debug info ausgeben.
- void [LogPositions](#) ([PosState](#) state)
Datensätze ausgeben.
- [Position operator\[\]](#) (int index)
[Position](#) gemäße index wiedergeben.

Private Methoden

- int8_t [RangeCheck](#) ()
Position Array Bereichs abfrage.
- [sPosition TosPosition](#) ([Position](#) ps)
Umwandlung des structs von float to Short int.
- [sPosture TosPosture](#) ([Posture](#) ps)
float to Short Int
- [PosState Decode](#) (String content, char delimiter=',')
Positions zustand Validieren und Ermitteln.
- void [Update](#) (int index, String ¶ms, [PosState](#) state)
Positionsliste Aktualisieren.

Private Attribute

- [RobFrame](#) * [_robFrame](#)
Zeiger Zur [RobFrame](#).
- [PosData](#) * [_data](#)
Array von Positions Daten.
- int [_definedCount](#) = 0
Anzahl definierte Datensätze innerhalb des Arrays.

5.11.1 Beschreibung der Konstruktoren und Destruktoren

5.11.1.1 PositionFactory()

```
PositionFactory::PositionFactory ()
```

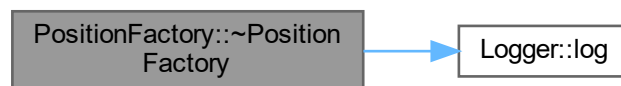
Klassen-Objekt erstellen.

5.11.1.2 ~PositionFactory()

```
PositionFactory::~~PositionFactory ()
```

Datenarray Vernichten.

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.11.2 Dokumentation der Elementfunktionen

5.11.2.1 AddPosition()

```
int & PositionFactory::AddPosition (
    String content,
    PosState state,
    char filter = ' ')

```

[Position](#), Winkelstellung hinzufügen.

Neuen Datensatz im allozierten Datenarray belegen.

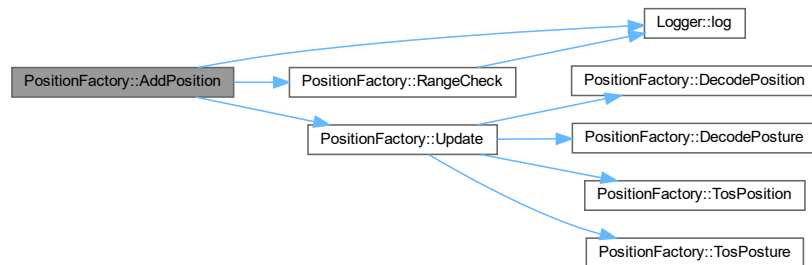
Parameter

	<i>content</i>	String
	<i>state</i>	Type
	<i>filter</i>	Trennzeichen

Rückgabe

index des hinzugefügte [Position](#)

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:

**5.11.2.2 Count()**

```
int & PositionFactory::Count ()
```

Datensatzanzahl.

Anzahl belegte Speicherplaeetze wiedergeben.

5.11.2.3 Decode()

```
PosState PositionFactory::Decode (
    String content,
    char delimiter = ',') [private]
```

Positions zustand Validieren und Ermitteln.

Parameter

<code>content</code>	String Position
<code>delimiter</code>	variable Trennzeichen

Rückgabe

Positionszustand

5.11.2.4 DecodePosition()

```
Position PositionFactory::DecodePosition (
    String input,
    String filter = ",")
```

String to [Position](#).

String Array in Positions-Struktur Transformieren.

Parameter

<i>input</i>	string
<i>filter</i>	Trennzeichen

Rückgabe

Position

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.11.2.5 DecodePosture()

```
Posture PositionFactory::DecodePosture (  
    String input,  
    String filter = ",")
```

String to Position.

String Array in Winkelstellungs-Struktur Transformieren.

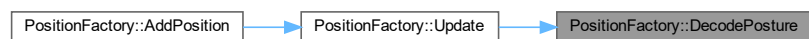
Parameter

<i>input</i>	string
<i>filter</i>	Trennzeichen

Rückgabe

Winkelstellung

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.11.2.6 Del()

```
void PositionFactory::Del (  
    String identifier)
```

nicht Implementiert

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.11.2.7 Get()

```
void PositionFactory::Get (  
    String identifier)
```

nicht Implementiert

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.11.2.8 Init()

```
void PositionFactory::Init (  
    RobFrame * robFrame)
```

RobFrameWork Zeigerzuweisung und Speicherplatz allozieren.

Initialisierung Speicher Allokation

Parameter

<u>robFrame</u>	Zeiger
-----------------	--------

Rückgabe

void

5.11.2.9 LogInfo()

```
void PositionFactory::LogInfo ()
```

Debug info ausgeben.

Debug Daten ausdrucken. Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.11.2.10 LogPositions()

```
void PositionFactory::LogPositions (  
    PosState state)
```

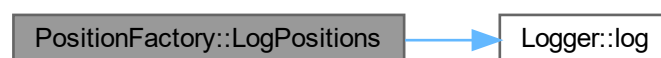
Datensätze ausgeben.

Datensätze gemäß Type zum Serial Ausgeben.

Parameter

Type

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.11.2.11 operator[]()

```
Position PositionFactory::operator[] (
    int index)
```

Position gemaeße index wiedergeben.

Datensatz index wiedergeben.

Rückgabe

Position

5.11.2.12 RangeCheck()

```
int8_t PositionFactory::RangeCheck () [private]
```

Postion Array Bereichs abfrage.

Rückgabe

1 wenn nicht voll -1 wenn voll

Pruft ob die Menge von Positionen Übersritten wurden ist Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.11.2.13 Setup()

```
void PositionFactory::Setup ()
```

Initialisierung.

Allozierte Speicherplatz validieren. Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.11.2.14 TosPosition()

```
sPosition PositionFactory::TosPosition (
    Position ps) [private]
```

Umwandlung des structs von float to Short int.

@breif float to Short Int

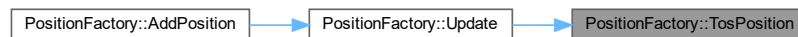
Parameter

<code>ps</code>	<code>Position</code>
-----------------	-----------------------

Rückgabe

Short Positionen

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.11.2.15 TosPosture()

```
sPosture PositionFactory::TosPosture (
    Posture ps) [private]
```

float to Short Int

Umwandlung des structs von float to Short Int.

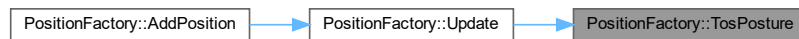
Parameter

<code>ps</code>	Winkelstellung
-----------------	----------------

Rückgabe

Short Winkelstellungen

Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**5.11.2.16 Update()**

```

void PositionFactory::Update (
    int index,
    String & params,
    PosState state) [private]
  
```

Positionsliste Aktualisieren.

Vorhandene Datensätze Aktualisieren.

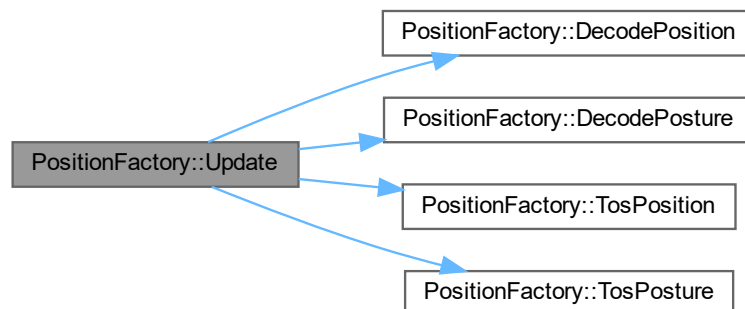
Parameter

<code>index</code>	Datensatzes
<code>params</code>	String Datensatz @state param Positions Type

Rückgabe

void

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.11.3 Dokumentation der Datenelemente

5.11.3.1 `_data`

```
PosData* PositionFactory::_data [private]
```

Array von Positions Daten.

5.11.3.2 `_definedCount`

```
int PositionFactory::_definedCount = 0 [private]
```

Anzahl definierte Datensätze innerhalb des Arrays.

5.11.3.3 `_robFrame`

```
RobFrame* PositionFactory::_robFrame [private]
```

Zeiger Zur [RobFrame](#).

Die Dokumentation für diese Klasse wurde erzeugt aufgrund der Dateien:

- [PositionFactory.hpp](#)
- [PositionFactory.cpp](#)

5.12 Posture Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- float [Jt1](#)
- float [Jt2](#)
- float [Jt3](#)
- float [Jt4](#)
- float [Jt5](#)
- float [Jt6](#)

5.12.1 Dokumentation der Datenelemente

5.12.1.1 Jt1

```
float Posture::Jt1
```

5.12.1.2 Jt2

```
float Posture::Jt2
```

5.12.1.3 Jt3

```
float Posture::Jt3
```

5.12.1.4 Jt4

```
float Posture::Jt4
```

5.12.1.5 Jt5

```
float Posture::Jt5
```

5.12.1.6 Jt6

```
float Posture::Jt6
```

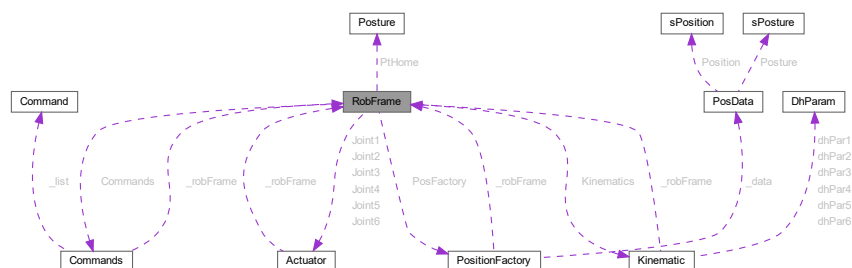
Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.13 RobFrame Klassenreferenz

```
#include <RobFrameWork.hpp>
```

Zusammengehörigkeiten von RobFrame:



Öffentliche Methoden

- [RobFrame](#) ()
Klassen-Objekt.
- void [Setup](#) ()
Einrichtung und Speicherallokation.
- void [Decode](#) (String data)
Eingangsdaten Parsen.
- void [GotoDeg](#) ([Actuator](#) &jt, int dest)
Glenk Einzelverfahren.
- void [LoopPositions](#) ()
Iteriere durch den Positionsdatensatz.
- [PosState Drive](#) ([Posture](#) pt)
Punkt zu Punkt Asynchrone Gelenk-Ansteuerung.
- [PosState Drive](#) ([Position](#) ps)
Punkt zu Punkt Asynchrone Gelenk-Ansteuerung.
- [PosState LDrive](#) ([Position](#) ps)
Gradlinige Asynchrone Gelenk-Ansteuerung.

Öffentliche Attribute

- [Posture PtHome](#) {0, 45, -90, 0, -45, 0}
Grundstellung.
- [Commands Commands](#)
Befehls-Manager.
- [Kinematic Kinematics](#)
Kinematik-Manager.
- [PositionFactory PosFactory](#)
Position-Manager.
- [Actuator Joint1](#)
Gelenk 1.
- [Actuator Joint2](#)
Gelenk 2.
- [Actuator Joint3](#)
Gelenk 3.
- [Actuator Joint4](#)
Gelenk 4.
- [Actuator Joint5](#)
Gelenk 5.
- [Actuator Joint6](#)
Gelenk 6.

Private Methoden

- void [Increment](#) ([Actuator](#) &jt, int origin, int dest, int itr)
Aktor Asynchron ansteuern.
- void [Sweep](#) ([Actuator](#) &jt, int origin, int dest)
Aktor Synchron Ansteuern.
- void [Validate](#) ([Position](#) &origin, [Position](#) &dest, int &range, int &valid, [PosState](#) state=[PosState::PosAndPost](#))
Positions Soll, Ist-Positions vergleich.
- [Position Interpolate](#) ([Position](#) &origin, [Position](#) &dest, int range, int step=1)
Gradlinige Interpolation zwischen Zwei Punkte.

Private Attribute

- `int _loopCount = 0`
Schleifen zähler.

5.13.1 Beschreibung der Konstruktoren und Destruktoren

5.13.1.1 RobFrame()

```
RobFrame::RobFrame ()
```

Klassen-Objekt.

5.13.2 Dokumentation der Elementfunktionen

5.13.2.1 Decode()

```
void RobFrame::Decode (
    String data)
```

Eingangsdaten Parsen.

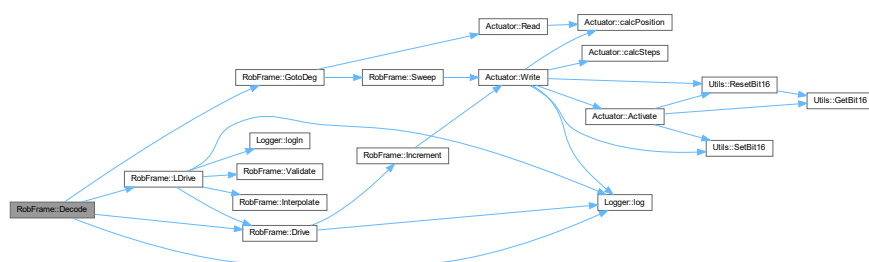
Parameter

<code>data</code>	String data
-------------------	-------------

Rückgabe

`void`

Eingangs Daten aus Serialle Port lesen Auswerten und entsprechend aus führen. Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.13.2.2 Drive() [1/2]

```
PosState RobFrame::Drive (
    Position ps)
```

Punkt zu Punkt Asynchrone Gelenk-Ansteuerung.

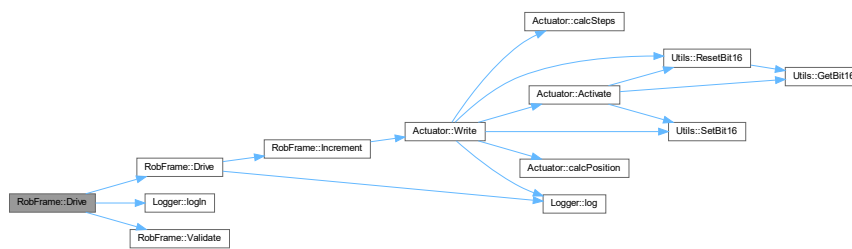
Parameter

<code>ps</code>	Soll Position
-----------------	----------------------

Rückgabe

Positions Status

Verfahre alle Gelenke Asynchron zum Ziel **Position** mit einer Point zu Punkt bzw. Gelenk Bewegung Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.13.2.3 Drive() [2/2]

```

PosState RobFrame::Drive (
    Posture pt)
  
```

Punkt zu Punkt Asynchrone Gelenk-Ansteuerung.

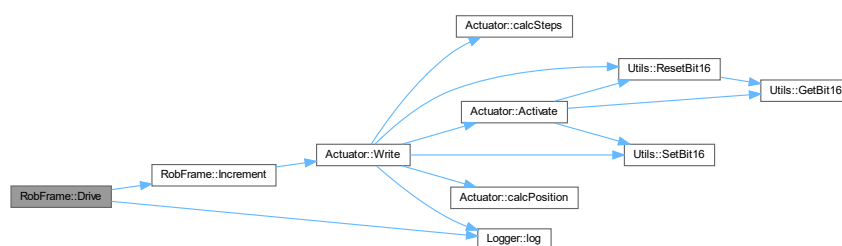
Parameter

<code>pt</code>	Soll Winkelstellung
-----------------	---------------------

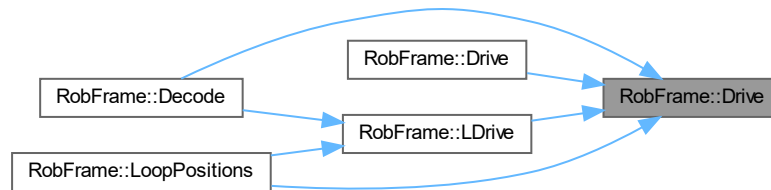
Rückgabe

Positions Status

Verfahre alle Gelenke Asynchrone zum Ziel Stellung in Punkt zu Punkt Bewegung Hier ist ein Graph, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.13.2.4 GotoDeg()

```
void RobFrame::GotoDeg (
    Actuator & jt,
    int dest)
```

Glenk Einzelverfahren.

Verfahre Gelenk Einzel zum Soll-Position.

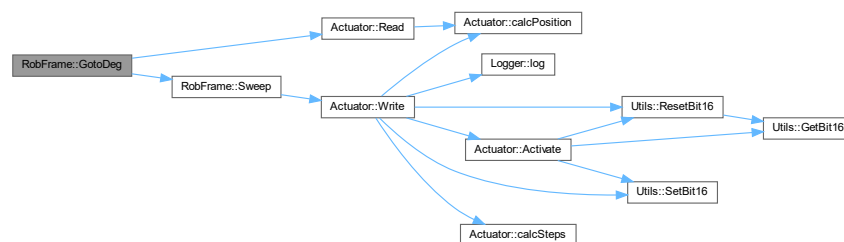
Parameter

<code>jt</code>	Aktuator Referenz
<code>dest</code>	Ziel-Position

Rückgabe

void

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.13.2.5 Increment()

```
void RobFrame::Increment (
    Actuator & jt,
    int origin,
    int dest,
    int itr) [private]
```

Aktor Asynchron ansteuern.

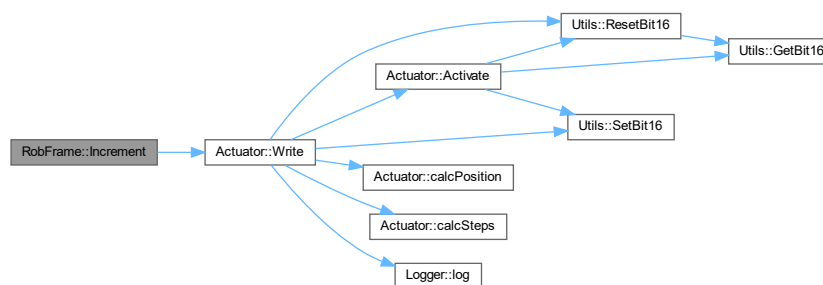
Parameter

	<i>jt</i>	Aktuator referenz
	<i>origin</i>	Ist-Position
	<i>dest</i>	Ziel-Position
	<i>itr</i>	Inkrementen Wert

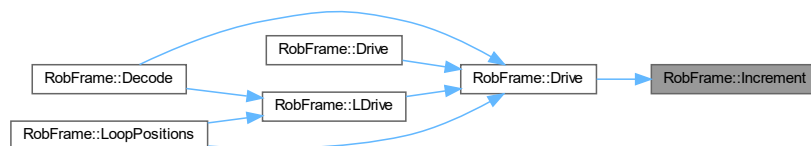
Rückgabe

void

Gelenk Aktorik schrittweise inkrementieren asynchrone Ansteuerung zur ermöglichen Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.13.2.6 Interpolate()

```
Position RobFrame::Interpolate (
    Position & origin,
    Position & dest,
    int range,
    int step = 1) [private]
```

Gradlinige Interpolation zwischen Zwei Punkte.

Gradlinige Interpolation zwischen zwei punkte $\text{delta} = \text{Ist-Position} + \text{Schritt} * (\text{sign}(\text{Richtung}))$ Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.13.2.7 LDrive()

```
PosState RobFrame::LDrive (
    Position ps)
```

Gradlinige Asynchrone Gelenk-Ansteuerung.

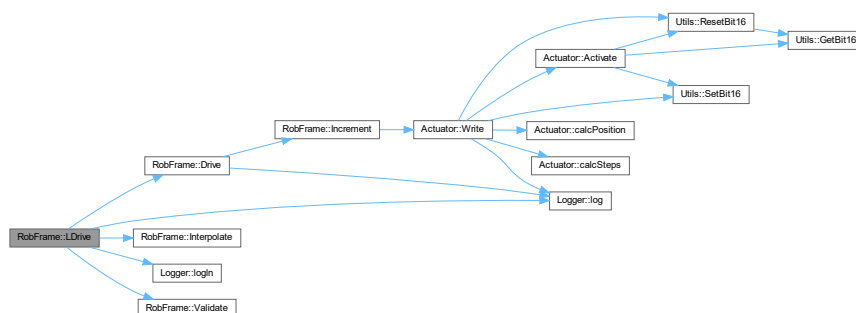
Parameter

<code>ps</code>	Soll Position
-----------------	-------------------------------

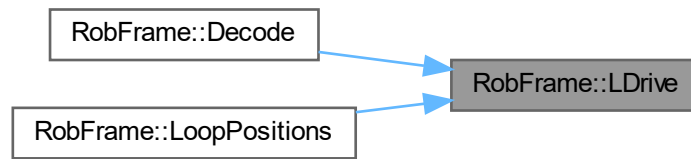
Rückgabe

Positions Status

Verfahre alle Gelenke Asynchron zum Ziel [Position](#) mit einer gradlinige Bewegung Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



5.13.2.8 LoopPositions()

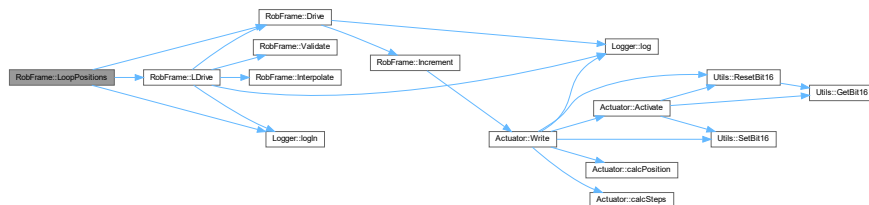
```
void RobFrame::LoopPositions ()
```

Iteriere durch den Positionsdatensatz.

Rückgabe

void

Iteriere durch den positions Datensatz Fahre in Punkt zu Punkt bzw. [Joint](#) Bewegung oder fahre Gradlinige Bewegung. Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



5.13.2.9 Setup()

```
void RobFrame::Setup ()
```

Einrichtung und Speicherallokation.

Einrichtung des Aktoren, und Positions Daten allozieren.

5.13.2.10 Sweep()

```
void RobFrame::Sweep (
    Actuator & jt,
    int origin,
    int dest) [private]
```

Aktor Synchron Ansteuern.

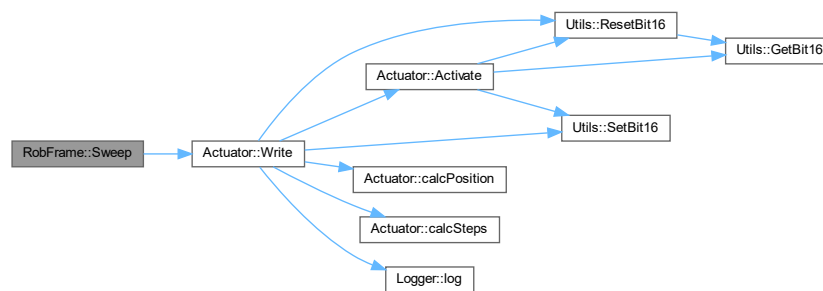
Parameter

	<i>it</i>	Aktuator referenz
	<i>origin</i>	Ist-Position
	<i>dest</i>	Ziel-Position

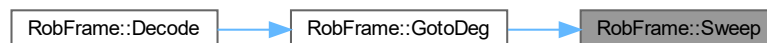
Rückgabe

void

Manipulator Aktorik synchron ansteuern d.h. mit Thread Sperrung Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**5.13.2.11 Validate()**

```

void RobFrame::Validate (
    Position & origin,
    Position & dest,
    int & range,
    int & valid,
    PosState state = PosState::PosAndPost) [private]
  
```

Positions Soll, Ist-Positions vergleich.

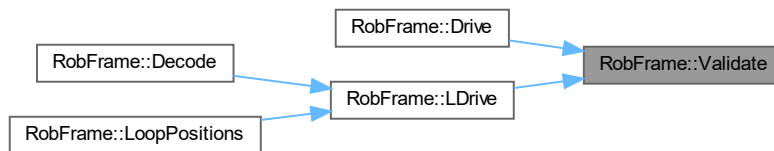
Parameter

	<i>origin</i>	Ist-Position
	<i>dest</i>	Soll-Position
	<i>range</i>	Tolerance
	<i>Flag</i>	Validierungs Merker 1 wenn Ist gleich Soll-Position, 0 wenn Ist ungleich Soll-Position.
	<i>state</i>	Datensatz Type

Rückgabe

void

Ist-Soll Positions Vergleich für die validierung der Inverskinematik Berechnung Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**5.13.3 Dokumentation der Datenelemente****5.13.3.1 _loopCount**

```
int RobFrame::_loopCount = 0 [private]
```

Schleifen zähler.

5.13.3.2 Commands

```
Commands RobFrame::Commands
```

Befehls-Manager.

5.13.3.3 Joint1

```
Actuator RobFrame::Joint1
```

Gelenk 1.

5.13.3.4 Joint2

```
Actuator RobFrame::Joint2
```

Gelenk 2.

5.13.3.5 Joint3

```
Actuator RobFrame::Joint3
```

Gelenk 3.

5.13.3.6 Joint4

`Actuator RobFrame::Joint4`

Gelenk 4.

5.13.3.7 Joint5

`Actuator RobFrame::Joint5`

Gelenk 5.

5.13.3.8 Joint6

`Actuator RobFrame::Joint6`

Gelenk 6.

5.13.3.9 Kinematics

`Kinematic RobFrame::Kinematics`

Kinematik-Manager.

5.13.3.10 PosFactory

`PositionFactory RobFrame::PosFactory`

Position-Manager.

5.13.3.11 PtHome

`Posture RobFrame::PtHome {0, 45, -90, 0, -45, 0}`

Grundstellung.

Die Dokumentation für diese Klasse wurde erzeugt aufgrund der Dateien:

- [RobFrameWork.hpp](#)
- [RobFrameWork.cpp](#)

5.14 sPosition Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- short [X](#)
- short [Y](#)
- short [Z](#)
- short [A](#)
- short [B](#)
- short [C](#)

5.14.1 Dokumentation der Datenelemente

5.14.1.1 A

```
short sPosition::A
```

5.14.1.2 B

```
short sPosition::B
```

5.14.1.3 C

```
short sPosition::C
```

5.14.1.4 X

```
short sPosition::X
```

5.14.1.5 Y

```
short sPosition::Y
```

5.14.1.6 Z

```
short sPosition::Z
```

Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

5.15 sPosture Strukturreferenz

```
#include <Types.hpp>
```

Öffentliche Attribute

- short [Jt1](#)
- short [Jt2](#)
- short [Jt3](#)
- short [Jt4](#)
- short [Jt5](#)
- short [Jt6](#)

5.15.1 Dokumentation der Datenelemente

5.15.1.1 Jt1

```
short sPosture::Jt1
```

5.15.1.2 Jt2

```
short sPosture::Jt2
```

5.15.1.3 Jt3

```
short sPosture::Jt3
```

5.15.1.4 Jt4

```
short sPosture::Jt4
```

5.15.1.5 Jt5

```
short sPosture::Jt5
```

5.15.1.6 Jt6

```
short sPosture::Jt6
```

Die Dokumentation für diese Struktur wurde erzeugt aufgrund der Datei:

- [Types.hpp](#)

Kapitel 6

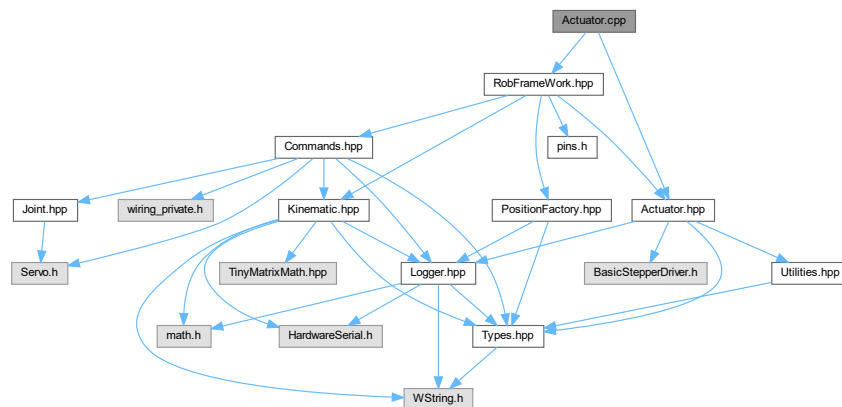
Datei-Dokumentation

6.1 Actuator.cpp-Dateireferenz

```
#include "RobFrameWork.hpp"
```

```
#include "Actuator.hpp"
```

Include-Abhängigkeitsdiagramm für Actuator.cpp:



6.2 Actuator.hpp-Dateireferenz

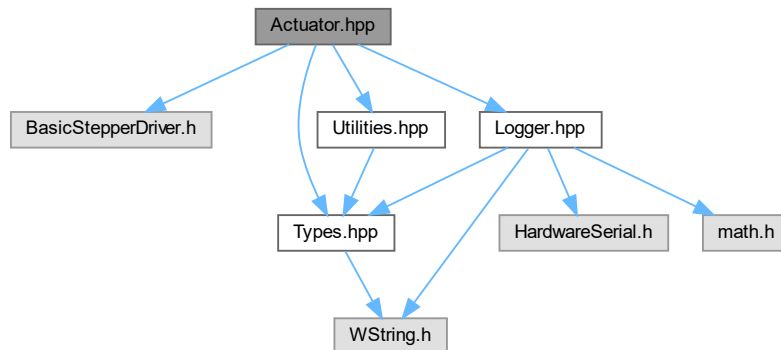
```
#include <BasicStepperDriver.h>
```

```
#include "Types.hpp"
```

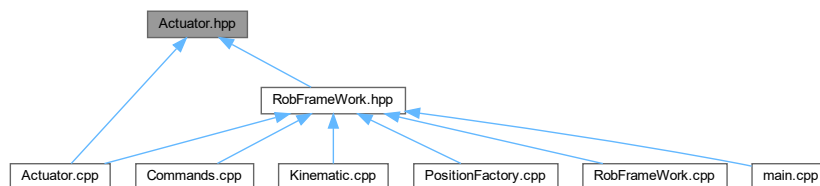
```
#include "Utilities.hpp"
```

```
#include "Logger.hpp"
```

Include-Abhängigkeitsdiagramm für Actuator.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- class [Actuator](#)

6.3 Actuator.hpp

[gehe zur Dokumentation dieser Datei](#)

```

00001
00003 // Datei: Actuator.hpp
00004 // Beschreibung: Ansteuerungslogik des Aktuators.
00005 //
00006 //           Dieser Klasse dient als wrappper für die
00007 //           "BasicStepperDriver" Klasss von Laurentiu Badea
00008 //           https://github.com/laurb9/StepperDriver
00009 //
00010 // Autor: Chukwunonso Bob-Anyeji
00011 // Datum: 18.08.2025@12-00
00012 // Aktualisiert: 27.08.2025@11-46
00014
00015 // NOTE: Motor steps per revolution.
00016 // die meisten Nema-17 Stepper haben 200 steps oder 1.8 degrees/step
00017 // mit 16 Miktrosteps Einstellung jenach Treiber
00018
00019 #ifndef ACTUATOR_H_
00020 #define ACTUATOR_H_
00021
  
```

```

00022 #include <BasicStepperDriver.h>
00023 #include "Types.hpp"
00024 #include "Utilities.hpp"
00025 #include "Logger.hpp"
00026
00027 class RobFrame;
00028 class Actuator
00029 {
00030     private:
00031         RobFrame* _robFrame;
00032         BasicStepperDriver* _stepper;
00033         // Stepper und Steppertreiber Einstellung
00034         uint8 _motorSteps = 200;
00035         uint8 _microSteps = 16;
00036         // Konfigurationswerte
00037         int8 _dir = 1;
00038         int16 _id = 0;
00039         int16 _offset;
00040         int16 _min = -90;
00041         int16 _max = 90;
00042         int16 _gearRatio = 1;
00043         int32 _previousSteps = 0;
00044         int32 _actualSteps = 0;
00045         int32 _targetSteps = 0;
00046         uint16 _timeOfDelay = 0;
00047         float _rpm = 10;
00048         // Zustands Variablen
00049         int8 _enabled = -1;
00050         word _state = 0;
00051         int32 calcSteps(int32 target);
00052         int32 calcPosition(int32 actSteps);
00053     public:
00054         Actuator ();
00055         void Init(RobFrame* robFrame);
00056         int16 Id();
00057         int16 Setup(float gearRatio = 1, int16 accuracy = 5, int16 max = 90, int16 min = -90);
00058         int16 Attach(int16 dirPin, int16 stepPin, int16 enaPin);
00059         int16 Accuracy;
00060         int32 Activate(int8 ena, uint16 tof = 5000);
00061         int32 Reference(int32 value = 0);
00062         int32 Write(int32 target);
00063         int32 Read();
00064         int32 Delta(int32 target);
00065         enum State {Enabled, Stopped, InMotion, Referenced, InPosition, Aborted};
00066 };
00067
00068 #endif // ACTUATOR_H_

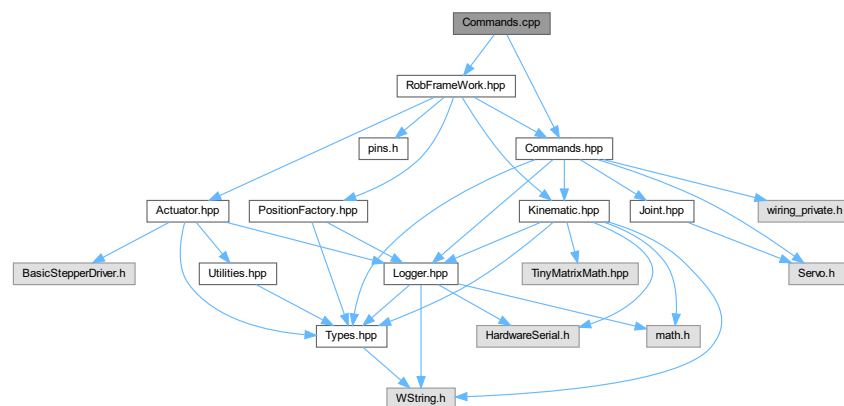
```

6.4 Commands.cpp-Dateireferenz

```
#include "RobFrameWork.hpp"
```

```
#include "Commands.hpp"
```

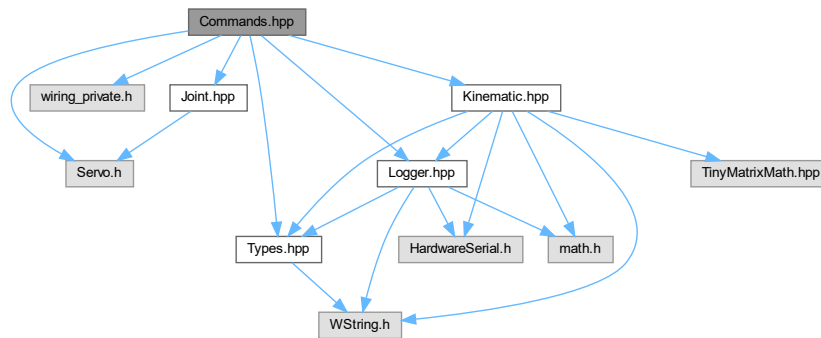
Include-Abhängigkeitsdiagramm für Commands.cpp:



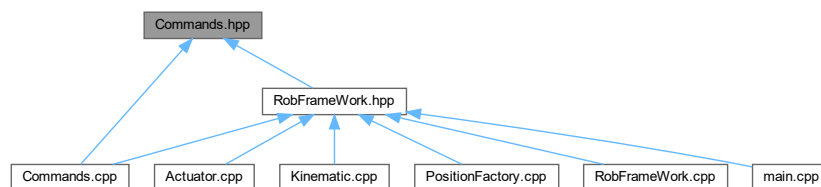
6.5 Commands.hpp-Dateireferenz

```
#include <Servo.h>
#include <wiring_private.h>
#include "Types.hpp"
#include "Logger.hpp"
#include "Joint.hpp"
#include "Kinematic.hpp"
```

Include-Abhängigkeitsdiagramm für Commands.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- class [Commands](#)

Makrodefinitionen

- #define [COMMAND_LEN](#) 28

6.5.1 Makro-Dokumentation

6.5.1.1 COMMAND_LEN

```
#define COMMAND_LEN 28
```

6.6 Commands.hpp

[gehe zur Dokumentation dieser Datei](#)

```

00001 /*
00002  * Program: Commands.hpp
00003  * Description: header file containing procedures
00004  *             for Parsing and validating Commands
00005  * Autor: Chukwunonso Bob-Anyeji
00006  * Date: 09.06.2024
00007  */
00008
00009 #ifndef COMMANDS_H_
00010 #define COMMANDS_H_
00011
00012 #define COMMAND_LEN 28
00013
00014 #include <Servo.h>
00015 #include <wiring_private.h>
00016
00017 #include "Types.hpp"
00018 #include "Logger.hpp"
00019 #include "Joint.hpp"
00020 #include "Kinematic.hpp"
00021
00022 class RobFrame;
00023 class Commands
00024 {
00025 private:
00026     int _range = 0;
00027     RobFrame* _robFrame;
00028     Command _list[COMMAND_LEN];
00029
00030 public:
00031     Commands();
00032
00033     // Const Difinitions
00034     enum Tags{
00035         /*0 */ World,
00036         /*1 */ // Read Set Joint Positions
00037         /*2 */ Jt1,
00038         /*3 */ Jt2,
00039         /*4 */ Jt3,
00040         /*5 */ Jt4,
00041         /*6 */ Jt5,
00042         /*7 */ Jt6,
00043         /*8 */ Jt7,
00044         /*9 */ // Get Rob Position Data
00045         /*10*/ Here,
00046         /*11*/ Joints,
00047         /*12*/ // Drive to Given Configurations
00048         /*13*/ GoHome,
00049         /*14*/ Drive,
00050         /*15*/ JDrive,
00051         /*16*/ LDrive,
00052         /*17*/ // Read Set Parameters
00053         /*18*/ Speed,
00054         /*19*/ Accuracy,
00055         /*20*/ Debug,
00056         /*21*/ Voltage,
00057         /*22*/ // Assignment and listing of Position Data
00058         /*23*/ Position,
00059         /*24*/ Posture,
00060         /*25*/ // Listing of Parameter and Data
00061         /*26*/ ListPositions,
00062         /*27*/ ListPostures,
00063         /*28*/ ListAllPositions,
00064         /*29*/ // Loops
00065         /*30*/ JLoop,
00066         /*31*/ LLoop,
00067         /*32*/ // Teach / Manual Functions
00068         /*33*/ Rotate,
00069         /*34*/ RotStep,
00070         /*35*/ // List and Document Command Parameters
00071         /*36*/ Help
00072     };
00073
00074     Command& operator[](int index);
00075
00076     Command Parse(String& input);
00077     Command Get(Tags tag);
00078
00079     void Init(RobFrame* robFrame);
00080
00081     int GetParam(int cIndex);
00082     void SetParam(int cIndex, float val);

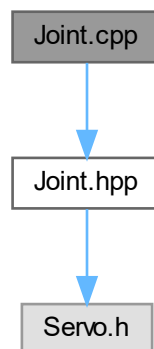
```

```
00083 void Delay();  
00084 void List();  
00085 };  
00086  
00087 #endif // COMMANDS_H_
```

6.7 Joint.cpp-Dateireferenz

```
#include "Joint.hpp"
```

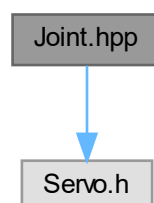
Include-Abhängigkeitsdiagramm für Joint.cpp:



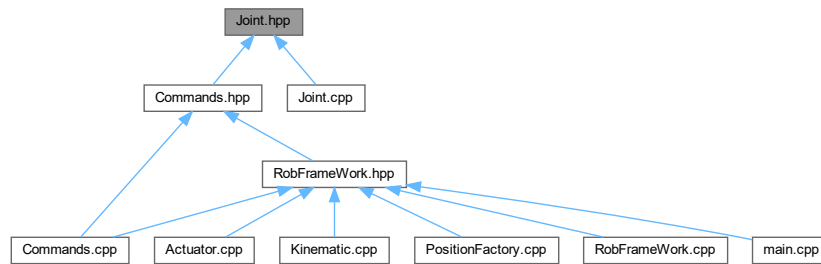
6.8 Joint.hpp-Dateireferenz

```
#include <Servo.h>
```

Include-Abhängigkeitsdiagramm für Joint.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- class [Joint](#)

6.9 Joint.hpp

[gehe zur Dokumentation dieser Datei](#)

```

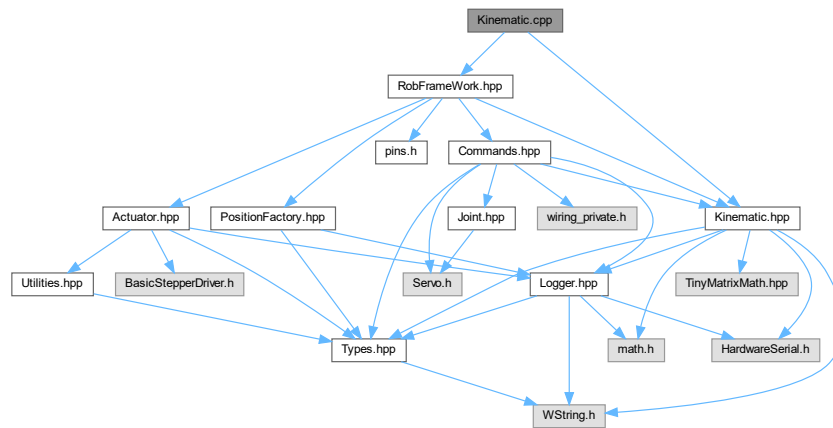
00001 /*
00002  * Program: Joint.hpp
00003  * Description: header file with wrapper fuctions
00004  *             for Servo.h class object
00005  * Autor: Chukwunonso Bob-Anyeji
00006  * Date: 09.06.2024
00007  */
00008
00009 #ifndef JOINT_H_
00010 #define JOINT_H_
00011
00012 #include <Servo.h>
00013
00014 class RobFrame;
00015 class Joint
00016 {
00017 private:
00018     int _offset;
00019     RobFrame* _robFrame;
00020 public:
00021     Joint();
00022
00023     void Init(RobFrame* robFrame);
00024
00025     void write(int degree);
00026     void attach(int pin);
00027     void attach(int pin, int min, int max);
00028     void attach(int pin, int min, int max, int offset);
00029     void limit(int min, int max);
00030     int read();
00031     int read2();
00032     int readMicroseconds();
00033     int delta(int dest);
00034
00035     int id = 0;
00036     int offset = 0;
00037     int min = 1;
00038     int max = 180;
00039     Servo servo;
00040 };
00041
00042 #endif // JOINT_H_
  
```

6.10 Kinematic.cpp-Dateireferenz

```
#include "RobFrameWork.hpp"
```

```
#include "Kinematic.hpp"
```

Include-Abhängigkeitsdiagramm für Kinematic.cpp:



Funktionen

- float **getRad** (float deg)
Grad zu Radiant konvertieren.
- float **getDeg** (float rad)
Radiant zu Grad konvertieren.
- float **sign** (float val)
Signal Wert des Eingangs wiedergeben.

Variablen

- const float **pi** = 3.1416
pi Definition

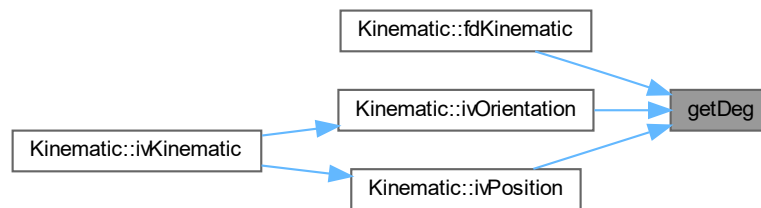
6.10.1 Dokumentation der Funktionen

6.10.1.1 getDeg()

```
float getDeg (
    float rad)
```

Radiant zu Grad konvertieren.

Radiant zu Grad. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

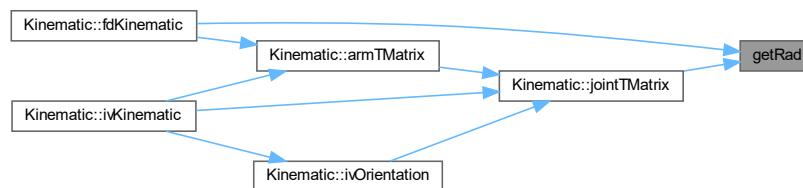


6.10.1.2 getRad()

```
float getRad (
    float deg)
```

Grad zu Radiant konvertieren.

Grad zu Radiant. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



6.10.1.3 sign()

```
float sign (
    float val)
```

Signal Wert des Eingangs wiedergeben.

Signum Funktion. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



6.10.2 Variablen-Dokumentation

6.10.2.1 pi

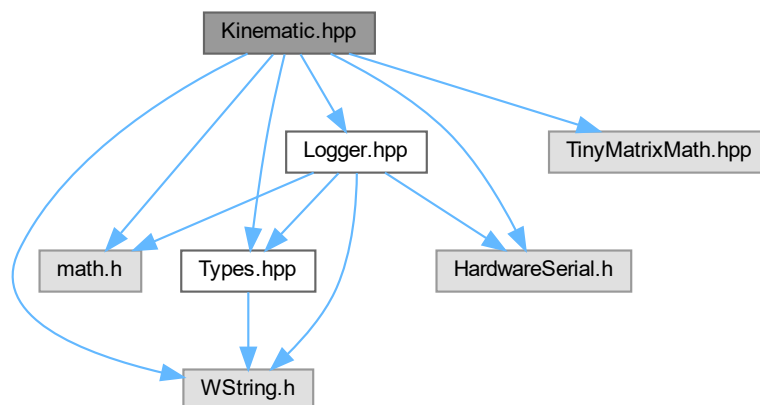
```
const float pi = 3.1416
```

pi Definition

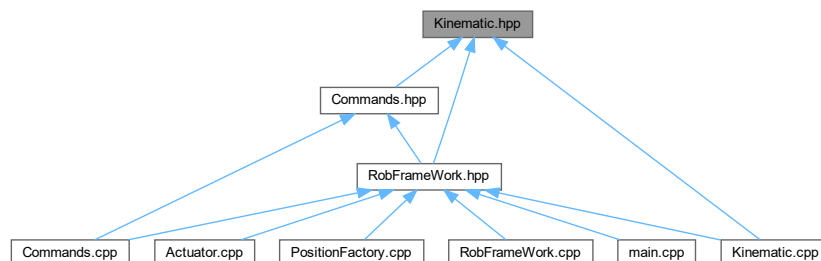
6.11 Kinematic.hpp-Dateireferenz

```
#include <math.h>
#include <WString.h>
#include <HardwareSerial.h>
#include <TinyMatrixMath.hpp>
#include "Types.hpp"
#include "Logger.hpp"
```

Include-Abhängigkeitsdiagramm für Kinematic.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- class [Kinematic](#)

Funktionen

- float [getRad](#) (float deg)
Grad zu Radian.
- float [getDeg](#) (float rad)
Radian zu Grad.
- float [sign](#) (float value)
Signum Funktion.

6.11.1 Dokumentation der Funktionen**6.11.1.1 getDeg()**

```
float getDeg (
    float rad)
```

Radian zu Grad.

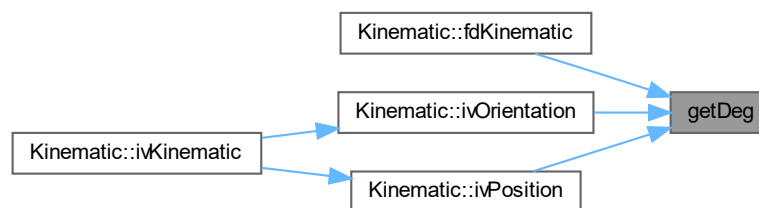
Parameter

<i>rad</i>	Radian
------------	--------

Rückgabe

Grad

Radian zu Grad. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**6.11.1.2 getRad()**

```
float getRad (
    float deg)
```

Grad zu Radian.

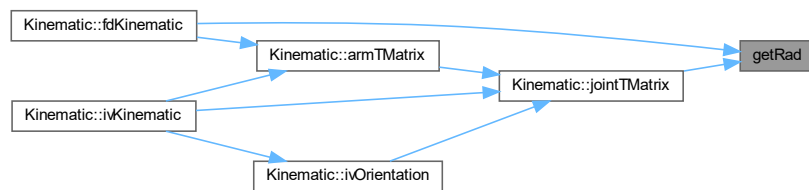
Parameter

<code>deg</code>	Grad wert
------------------	-----------

Rückgabe

Radiant

Grad zu Radiant. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:

**6.11.1.3 sign()**

```
float sign (
    float val)
```

Signum Funktion.

Parameter

<code>value</code>

Rückgabe

1 oder -1

Signum Funktion. Hier ist ein Graph der zeigt, wo diese Funktion aufgerufen wird:



6.12 Kinematic.hpp

[gehe zur Dokumentation dieser Datei](#)

```

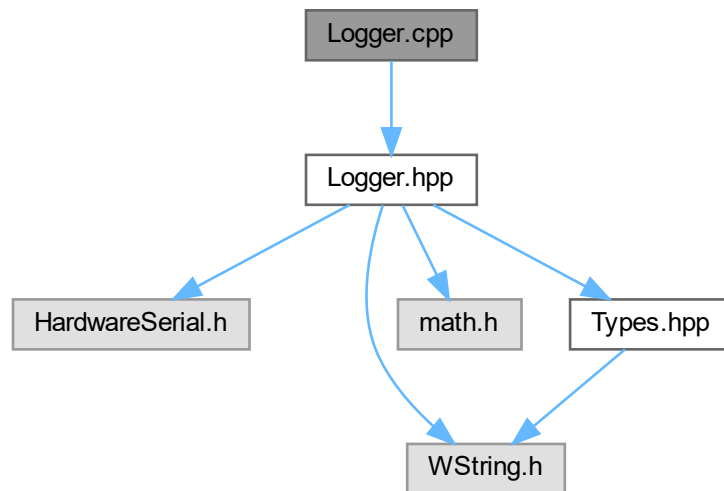
00001
00003 // Datei: Kinematic.hpp
00004 //
00005 // Beschreibung: Berechnungen und Prozeduren zur Ermittlung der Vor- und
00006 //               Inverskinematik.
00007 //
00008 //               Dieser Klassier die TinyMatrixMath Klasse von "Micah Mundy"
00009 //               https://github.com/m516/TinyMatrixMath?tab=MIT-1-ov-file
00010 //               zu handhaben von Matrizen
00011 //
00012 // Autor: Chukwunonso Bob-Anyeji
00013 // Datum: 09.06.2024@12-00
00014 // Aktualisiert: 27.08.2025@16-54
00016
00017 #ifndef KINEMATIC_H_
00018 #define KINEMATIC_H_
00019
00020 #include <math.h>
00021 #include <WString.h>
00022 #include <HardwareSerial.h>
00023 #include <TinyMatrixMath.hpp>
00024
00025 #include "Types.hpp"
00026 #include "Logger.hpp"
00027
00031 float getRad(float deg);
00032
00036 float getDeg(float rad);
00037
00041 float sign(float value);
00042
00043 class RobFrame;
00044 class Kinematic
00045 {
00046     private:
00048         RobFrame* _robFrame;
00052         tmm::Matrix<4,4> jointTMatrix(DhParam dhPar);
00056         tmm::Matrix<4,4> armTMatrix(Posture posture);
00059         tmm::Matrix<3,3> read3x3(tmm::Matrix<4,4> tMat);
00060     public:
00062         Kinematic();
00066         void Init(RobFrame* robFrame);
00071         void printMatrix(tmm::Matrix<4,4> mat, String heading);
00075         Position fdKinematic(Posture posture);
00079         Posture ivKinematic(Position position);
00083         Posture ivPosition(Position position);
00088         Posture ivOrientation(Posture posture, Position ps);
00089
00091         DhParam dhPar1 = {};
00093         DhParam dhPar2 = {};
00095         DhParam dhPar3 = {};
00097         DhParam dhPar4 = {};
00099         DhParam dhPar5 = {};
00101         DhParam dhPar6 = {};
00102
00104         tmm::Matrix<4,4> tMat01 = {};
00106         tmm::Matrix<4,4> tMat12 = {};
00108         tmm::Matrix<4,4> tMat23 = {};
00110         tmm::Matrix<4,4> tMat34 = {};
00112         tmm::Matrix<4,4> tMat45 = {};
00114         tmm::Matrix<4,4> tMat56 = {};
00115
00117         tmm::Matrix<4,4> tMat02 = {};
00119         tmm::Matrix<4,4> tMat03 = {};
00121         tmm::Matrix<4,4> tMat04 = {};
00123         tmm::Matrix<4,4> tMat05 = {};
00125         tmm::Matrix<4,4> tMat06 = {};
00126 };
00127
00128 #endif // KINEMATIC_H_

```

6.13 Logger.cpp-Dateireferenz

```
#include "Logger.hpp"
```

Include-Abhängigkeitsdiagramm für Logger.cpp:



Namensbereiche

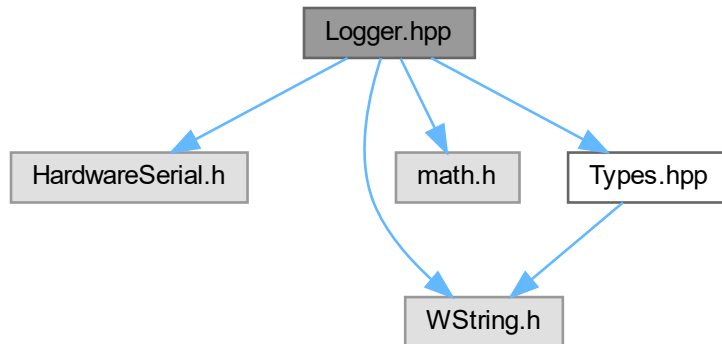
- namespace [Logger](#)

Funktionen

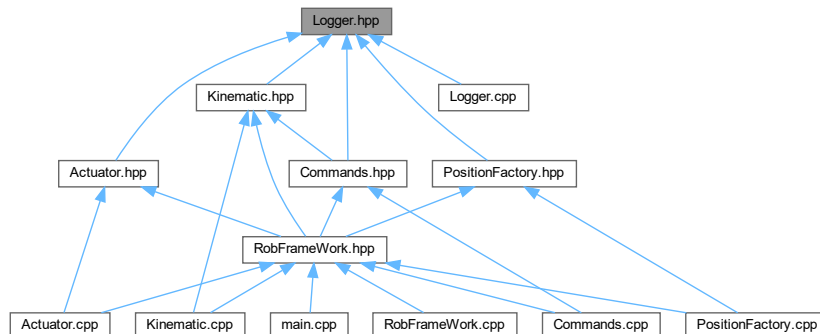
- void [Logger::logInit](#) (long baudRate)
- int [Logger::Incoming](#) ()
- void [Logger::encode](#) (String data)
- void [Logger::log](#) (String text="")
- void [Logger::logInt](#) (int digit=-1)
- void [Logger::logft](#) (float dec=-1)
- void [Logger::log](#) (String text="", String titel="")
- void [Logger::log](#) (int digit=-1, String titel="")
- void [Logger::logft](#) (float dec=-1.0, String titel="")
- void [Logger::log](#) ([Posture](#) pt)
- void [Logger::logIn](#) ([Posture](#) pt)
- void [Logger::logIn](#) ([Posture](#) pt, String title)
- String [Logger::ptEncode](#) (float val)
- void [Logger::logToSerial](#) ([Posture](#) pt)
- void [Logger::logToSerial](#) (String input)
- void [Logger::log](#) ([Position](#) ps)
- void [Logger::logIn](#) ([Position](#) ps)
- void [Logger::log](#) ([Position](#) ps, String title="Location and Orientation of Endeffector")
- void [Logger::logIn](#) ([Position](#) ps, String title)
- void [Logger::log](#) ([Command](#) cmd)
- void [Logger::log](#) ([PosData](#) pdata, [PosState](#) state)

6.14 Logger.hpp-Dateireferenz

```
#include <HardwareSerial.h>
#include <WString.h>
#include <math.h>
#include "Types.hpp"
Include-Abhängigkeitsdiagramm für Logger.hpp:
```



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Namensbereiche

- namespace [Logger](#)

Funktionen

- void [Logger::logInit](#) (long baudRate)
- int [Logger::Incoming](#) ()
- void [Logger::log](#) (String text="")

- void `Logger::logft` (float dec=-1)
- void `Logger::logInt` (int digit=-1)
- void `Logger::log` (String text="", String titel="")
- void `Logger::logft` (float dec=-1.0, String titel="")
- void `Logger::log` (int digit=-1, String titel="")
- void `Logger::log` (`Posture` pt)
- void `Logger::logIn` (`Posture` pt)
- void `Logger::logIn` (`Posture` pt, String title)
- void `Logger::logToSerial` (`Posture` pt)
- void `Logger::logToSerial` (String input)
- void `Logger::log` (`Position` ps)
- void `Logger::logIn` (`Position` ps)
- void `Logger::log` (`Position` ps, String title="Location and Orientation of Endeffector")
- void `Logger::logIn` (`Position` ps, String title)
- void `Logger::log` (`Command` cmd)
- void `Logger::log` (`PosData` pdata, `PosState` state)

6.15 Logger.hpp

[gehe zur Dokumentation dieser Datei](#)

```

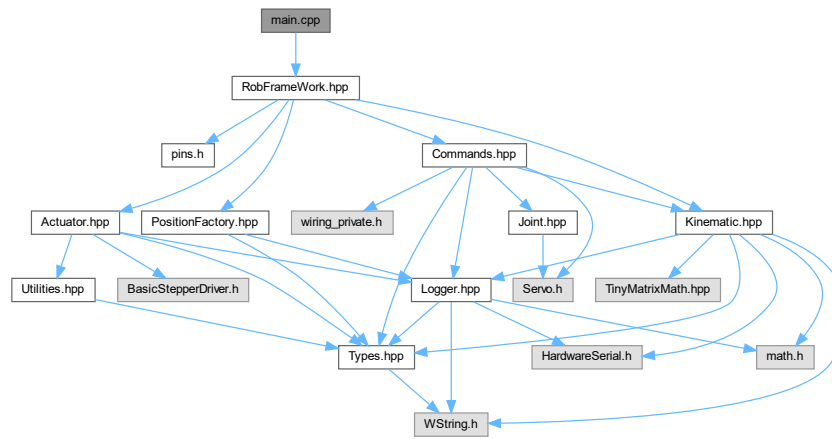
00001 /*
00002  * Program: Logger.hpp
00003  * Description: header file containing procedures for formatting and writing
00004  *              State Data to Serial port
00005  * Autor: Chukwunonso Bob-Anyeji
00006  * Date: 09.06.2024
00007  */
00008
00009 #ifndef LOGGER_H_
00010 #define LOGGER_H_
00011
00012 #include <HardwareSerial.h>
00013 #include <WString.h>
00014 #include <math.h>
00015 #include "Types.hpp"
00016
00017 /*
00018  * Current Naming convention clashes with logarithmic
00019  * functions from <math.h> C-header files ie.
00020  * => log(double arg);
00021  * => logf(float arg);
00022  * => logl(long double arg);
00023  * Todo:
00024  * + Update Encapsulate the below logging functions in
00025  *   separate namespace.
00026  */
00027
00028
00029 namespace Logger
00030 {
00031     void logInit(long baudRate);
00032     int Incoming();
00033     void log(String text);
00034     void logft(float dec);
00035     void logInt(int digit);
00036     void log(String text, String title);
00037     void logft(float dec, String title);
00038     void log(int digit, String titel);
00039     void log(Posture pt);
00040     void logIn(Posture pt);
00041     void logIn(Posture pt, String title);
00042     void logToSerial(Posture pt);
00043     void logToSerial(String input);
00044     void log(Position ps);
00045     void logIn(Position ps);
00046     void log(Position ps, String title);
00047     void logIn(Position ps, String title);
00048     void log(Command cmd);
00049     void log(PosData pdata, PosState state);
00050 };
00051 #endif // LOGGER_H_

```

6.16 main.cpp-Dateireferenz

```
#include "RobFrameWork.hpp"
```

Include-Abhängigkeitsdiagramm für main.cpp:



Funktionen

- void `setup()`
- void `loop()`

Variablen

- long `CycleCount` = 0
- `RobFrame` `RobArm`

6.16.1 Dokumentation der Funktionen

6.16.1.1 `loop()`

```
void loop ()
```

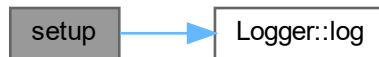
Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



6.16.1.2 setup()

```
void setup ()
```

Hier ist ein Graph, der zeigt, was diese Funktion aufruft:



6.16.2 Variablen-Dokumentation

6.16.2.1 CycleCount

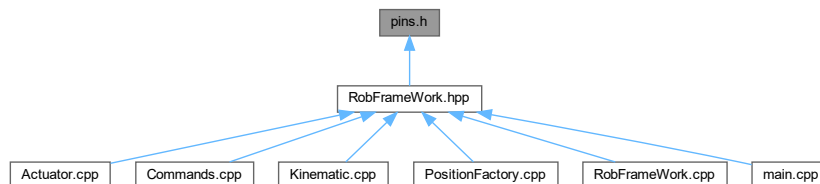
```
long CycleCount = 0
```

6.16.2.2 RobArm

```
RobFrame RobArm
```

6.17 pins.h-Dateireferenz

Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Makrodefinitionen

- `#define BOARD_INFO_NAME "RAMPS 1.4"`
- `#define SERVO1_PIN 11`
- `#define SERVO2_PIN 6`
- `#define SERVO3_PIN 5`
- `#define SERVO4_PIN 4`
- `#define JT1_LIM_PIN 3`
- `#define JT2_LIM_PIN 2`
- `#define JT3_LIM_PIN 14`
- `#define JT4_LIM_PIN 15`
- `#define JT5_LIM_PIN 18`
- `#define JT6_LIM_PIN 19`
- `#define JT1_STP_PIN 40`
- `#define JT1_DIR_PIN 42`
- `#define JT1_ENA_PIN 44`
- `#define JT2_STP_PIN 46`
- `#define JT2_DIR_PIN 48`
- `#define JT2_ENA_PIN 62`
- `#define JT3_STP_PIN 60`
- `#define JT3_DIR_PIN 61`
- `#define JT3_ENA_PIN 56`
- `#define JT4_STP_PIN 54`
- `#define JT4_DIR_PIN 55`
- `#define JT4_ENA_PIN 38`
- `#define JT5_STP_PIN 26`
- `#define JT5_DIR_PIN 28`
- `#define JT5_ENA_PIN 24`
- `#define JT6_STP_PIN 36`
- `#define JT6_DIR_PIN 34`
- `#define JT6_ENA_PIN 30`
- `#define ENC_INTP_ID 1`
- `#define ENC_CLK_PIN 3`
- `#define ENC_DIR_PIN 47`
- `#define ENC_BTN_PIN 45`

6.17.1 Makro-Dokumentation

6.17.1.1 BOARD_INFO_NAME

```
#define BOARD_INFO_NAME "RAMPS 1.4"
```

6.17.1.2 ENC_BTN_PIN

```
#define ENC_BTN_PIN 45
```

6.17.1.3 ENC_CLK_PIN

```
#define ENC_CLK_PIN 3
```

6.17.1.4 ENC_DIR_PIN

```
#define ENC_DIR_PIN 47
```

6.17.1.5 ENC_INTP_ID

```
#define ENC_INTP_ID 1
```

6.17.1.6 JT1_DIR_PIN

```
#define JT1_DIR_PIN 42
```

6.17.1.7 JT1_ENA_PIN

```
#define JT1_ENA_PIN 44
```

6.17.1.8 JT1_LIM_PIN

```
#define JT1_LIM_PIN 3
```

6.17.1.9 JT1_STP_PIN

```
#define JT1_STP_PIN 40
```

6.17.1.10 JT2_DIR_PIN

```
#define JT2_DIR_PIN 48
```

6.17.1.11 JT2_ENA_PIN

```
#define JT2_ENA_PIN 62
```

6.17.1.12 JT2_LIM_PIN

```
#define JT2_LIM_PIN 2
```

6.17.1.13 JT2_STP_PIN

```
#define JT2_STP_PIN 46
```

6.17.1.14 JT3_DIR_PIN

```
#define JT3_DIR_PIN 61
```

6.17.1.15 JT3_ENA_PIN

```
#define JT3_ENA_PIN 56
```

6.17.1.16 JT3_LIM_PIN

```
#define JT3_LIM_PIN 14
```

6.17.1.17 JT3_STP_PIN

```
#define JT3_STP_PIN 60
```

6.17.1.18 JT4_DIR_PIN

```
#define JT4_DIR_PIN 55
```

6.17.1.19 JT4_ENA_PIN

```
#define JT4_ENA_PIN 38
```

6.17.1.20 JT4_LIM_PIN

```
#define JT4_LIM_PIN 15
```

6.17.1.21 JT4_STP_PIN

```
#define JT4_STP_PIN 54
```

6.17.1.22 JT5_DIR_PIN

```
#define JT5_DIR_PIN 28
```

6.17.1.23 JT5_ENA_PIN

```
#define JT5_ENA_PIN 24
```

6.17.1.24 JT5_LIM_PIN

```
#define JT5_LIM_PIN 18
```

6.17.1.25 JT5_STP_PIN

```
#define JT5_STP_PIN 26
```

6.17.1.26 JT6_DIR_PIN

```
#define JT6_DIR_PIN 34
```

6.17.1.27 JT6_ENA_PIN

```
#define JT6_ENA_PIN 30
```

6.17.1.28 JT6_LIM_PIN

```
#define JT6_LIM_PIN 19
```

6.17.1.29 JT6_STP_PIN

```
#define JT6_STP_PIN 36
```

6.17.1.30 SERVO1_PIN

```
#define SERVO1_PIN 11
```

6.17.1.31 SERVO2_PIN

```
#define SERVO2_PIN 6
```

6.17.1.32 SERVO3_PIN

```
#define SERVO3_PIN 5
```

6.17.1.33 SERVO4_PIN

```
#define SERVO4_PIN 4
```

6.18 pins.h

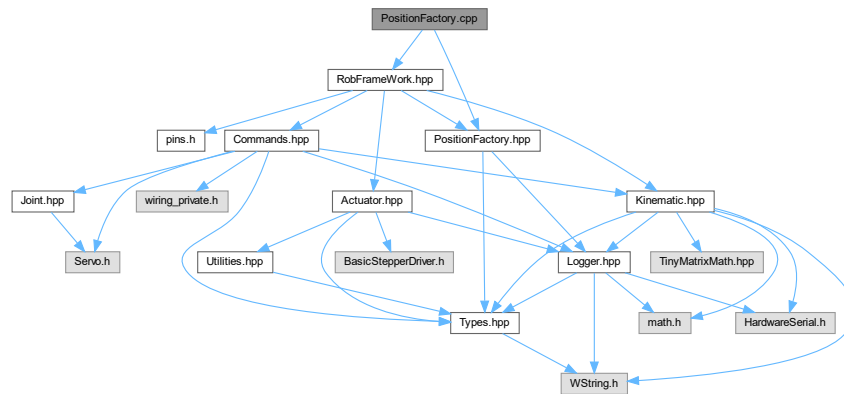
[gehe zur Dokumentation dieser Datei](#)

```
00001 /*
00002  * File: pins.h
00003  * Autor: Copywrite (c) Chukwunonso Bob-Anyeji
00004  * Description: Holds Pin definition for Ramps 1.4
00005  *             Shield on an Arduino Mega2560 Controller.
00006  * Date: 09.06.2024
00007  */
00008
00009 // Board Information
00010 #define BOARD_INFO_NAME "RAMPS 1.4"
00011
00012 // Servo Pins
00013 #define SERV01_PIN 11
00014 #define SERV02_PIN 6
00015 #define SERV03_PIN 5
00016 #define SERV04_PIN 4
00017
00018 // Limit Switchs
00019 #define JT1_LIM_PIN 3
00020 #define JT2_LIM_PIN 2
00021 #define JT3_LIM_PIN 14
00022 #define JT4_LIM_PIN 15
00023 #define JT5_LIM_PIN 18
00024 #define JT6_LIM_PIN 19
00025
00026 // Stepper Motor Pins
00027 #define JT1_STP_PIN 40
00028 #define JT1_DIR_PIN 42
00029 #define JT1_ENA_PIN 44
00030
00031 #define JT2_STP_PIN 46
00032 #define JT2_DIR_PIN 48
00033 #define JT2_ENA_PIN 62
00034
00035 #define JT3_STP_PIN 60
00036 #define JT3_DIR_PIN 61
00037 #define JT3_ENA_PIN 56
00038
00039 #define JT4_STP_PIN 54
00040 #define JT4_DIR_PIN 55
00041 #define JT4_ENA_PIN 38
00042
00043 #define JT5_STP_PIN 26
00044 #define JT5_DIR_PIN 28
00045 #define JT5_ENA_PIN 24
00046
00047 #define JT6_STP_PIN 36
00048 #define JT6_DIR_PIN 34
00049 #define JT6_ENA_PIN 30
00050
00051 // Rotary Encoder Pins
00052 #define ENC_INTP_ID 1
00053 #define ENC_CLK_PIN 3
00054 #define ENC_DIR_PIN 47
00055 #define ENC_BTN_PIN 45
```

6.19 PositionFactory.cpp-Dateireferenz

```
#include "RobFrameWork.hpp"
#include "PositionFactory.hpp"
```

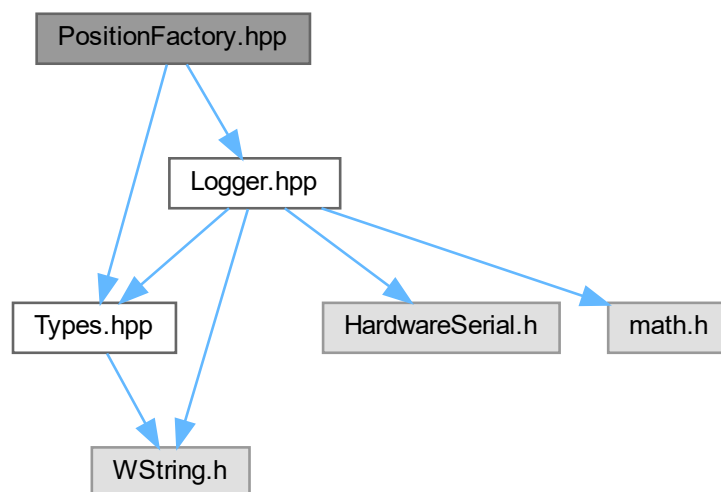
Include-Abhängigkeitsdiagramm für PositionFactory.cpp:



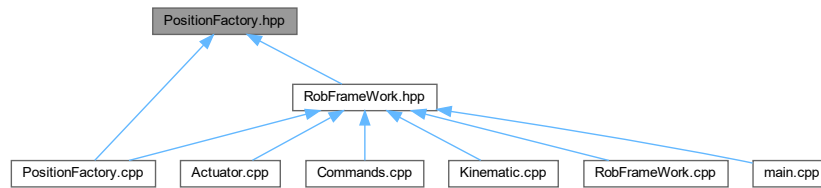
6.20 PositionFactory.hpp-Dateireferenz

```
#include "Types.hpp"
#include "Logger.hpp"
```

Include-Abhängigkeitsdiagramm für PositionFactory.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- class [PositionFactory](#)

Makrodefinitionen

- `#define POS_LIMIT 100`

6.20.1 Makro-Dokumentation

6.20.1.1 POS_LIMIT

```
#define POS_LIMIT 100
```

6.21 PositionFactory.hpp

[gehe zur Dokumentation dieser Datei](#)

```

00001
00003 // Datei: PositionFactory.hpp
00004 // Beschreibung: Dieser Klasse Beinhaltet Logik zur Erstellung, Vernichtung
00005 //                und Verwaltung von Positionen und Stellungen.
00006 //
00007 // Autor: Chukwunonso Bob-Anyejì
00008 // Datum: 11.07.2024@12-00
00009 // Aktualisiert: 27.08.2025@11-46
00011
00012 #ifndef POSITIONFACTORY_H_
00013 #define POSITIONFACTORY_H_
00014
00015 #define POS_LIMIT 100
00016
00017 #include "Types.hpp"
00018 #include "Logger.hpp"
00019
00020 class RobFrame;
00021 class PositionFactory
00022 {
00023     private:
00025         RobFrame* _robFrame;
00028         int8_t RangeCheck();
00032         sPosition TosPosition(Position ps);
00036         sPosture TosPosture(Posture ps);
00041         PosState Decode(String content, char delimiter = ',');
00047         void Update(int index, String& params, PosState state);
00049         PosData* _data;
00051         int _definedCount = 0;
00052     public:

```

```

00054     PositionFactory();
00056     ~PositionFactory();
00060     void Init(RobFrame* robFrame);
00062     void Setup();
00067     Posture DecodePosture(String input, String filter = ",");
00072     Position DecodePosition(String input, String filter = ",");
00078     int& AddPosition(String content, PosState state, char filter = ' ');
00080     int& Count();
00081     void Get(String identifier);
00082     void Del(String identifier);
00084     void LogInfo();
00087     void LogPositions(PosState state);
00090     Position operator[](int index);
00091 };
00092
00093 #endif // POSITIONFACTORY_H_

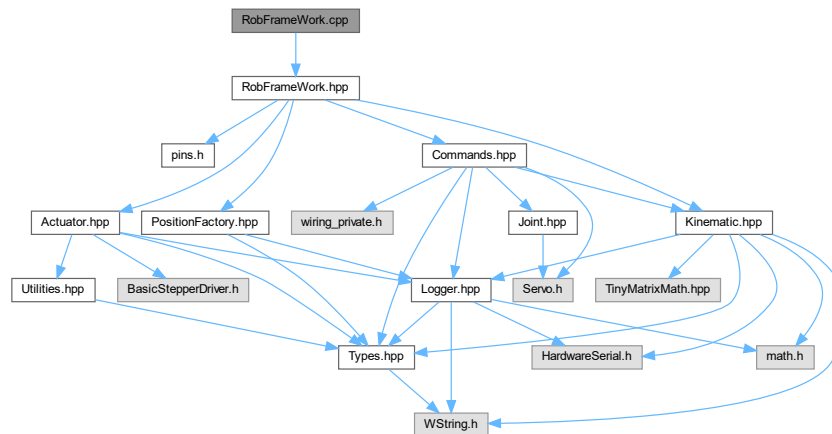
```

6.22 README.md-Dateireferenz

6.23 RobFrameWork.cpp-Dateireferenz

```
#include "RobFrameWork.hpp"
```

Include-Abhängigkeitsdiagramm für RobFrameWork.cpp:



6.24 RobFrameWork.hpp-Dateireferenz

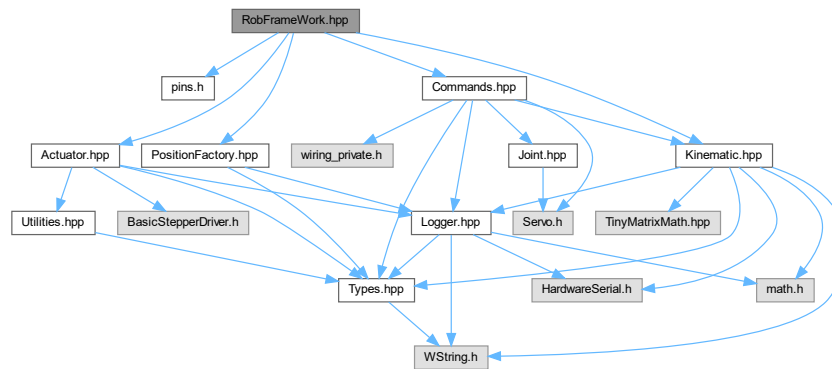
```

#include "pins.h"
#include "Actuator.hpp"
#include "Commands.hpp"
#include "Kinematic.hpp"

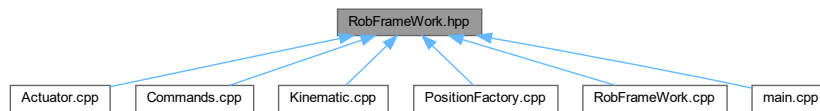
```

```
#include "PositionFactory.hpp"
```

Include-Abhängigkeitsdiagramm für RobFrameWork.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- class [RobFrame](#)

6.25 RobFrameWork.hpp

[gehe zur Dokumentation dieser Datei](#)

```
00001
00003 // File: RobFramWork.hpp
00004 // Beschreibung: Klassen Objekt zur Verwaltung, Steuerung und ausführung
00005 //               Des Manipulators
00006 // Autor: Chukwunonso Bob-Anyeji
00007 // Datum: 18.08.2025@12-00
00008 // Aktualisiert: 27.08.2025@11-46
00010
00011 #ifndef ROBFRAMEWORK_H_
00012 #define ROBFRAMEWORK_H_
00013
00014 #include "pins.h"
00015 #include "Actuator.hpp"
00016 #include "Commands.hpp"
00017 #include "Kinematic.hpp"
00018 #include "PositionFactory.hpp"
00019
00020 class RobFrame
00021 {
00022     private:
00024         int _loopCount = 0;
00031         void Increment(Actuator& jt, int origin, int dest, int itr);
00037         void Sweep(Actuator& jt, int origin, int dest);
```

```

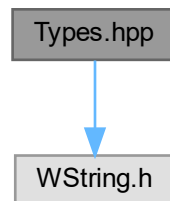
00046     void Validate(Position& origin, Position& dest, int& range, int& valid, PosState state =
PosState::PosAndPost);
00048     Position Interpolate(Position& origin, Position& dest, int range, int step = 1);
00049 public:
00051     RobFrame();
00053     Posture PtHome {0, 45, -90, 0, -45, 0};
00055     Commands Commands;
00057     Kinematic Kinematics;
00059     PositionFactory PosFactory;
00061     Actuator Joint1;
00063     Actuator Joint2;
00065     Actuator Joint3;
00067     Actuator Joint4;
00069     Actuator Joint5;
00071     Actuator Joint6;
00073     void Setup();
00077     void Decode(String data);
00082     void GotoDeg(Actuator& jt, int dest);
00085     void LoopPositions();
00089     PosState Drive(Posture pt);
00093     PosState Drive(Position ps);
00097     PosState LDrive(Position ps);
00098 };
00099
00100 #endif // ROBFRAMEWORK_H_

```

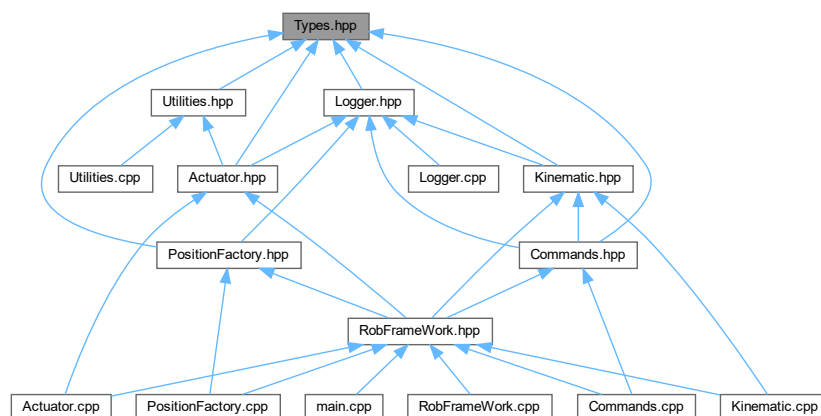
6.26 Types.hpp-Dateireferenz

```
#include <WString.h>
```

Include-Abhängigkeitsdiagramm für Types.hpp:



Dieser Graph zeigt, welche Datei direkt oder indirekt diese Datei enthält:



Klassen

- struct [DhParameters](#)
- struct [DhParam](#)
- struct [Matrix4x4](#)
- struct [Position](#)
- struct [sPosition](#)
- struct [Posture](#)
- struct [sPosture](#)
- struct [PosData](#)
- struct [Command](#)

Makrodefinitionen

- `#define IDENTIFIER_LEN 16`
- `#define OK 1`
- `#define NOK -1`
- `#define ON 1`
- `#define OFF 0`

Typdefinitionen

- using [int8](#) = signed char
- using [uint8](#) = unsigned char
- using [byte](#) = unsigned char
- using [int16](#) = short int
- using [uint16](#) = unsigned short int
- using [word](#) = unsigned int
- using [int32](#) = long int
- using [uint32](#) = unsigned long int
- using [int64](#) = long long int
- using [uint64](#) = unsigned long long int

Aufzählungen

- enum class [PosState](#) {
 [Invalid](#) = -1 , [Undefined](#) = 0 , [Defined](#) = 1 , [Position](#) = 4 ,
 [Posture](#) = 8 , [PosAndPost](#) = 16 }

6.26.1 Makro-Dokumentation

6.26.1.1 IDENTIFIER_LEN

```
#define IDENTIFIER_LEN 16
```

6.26.1.2 NOK

```
#define NOK -1
```

6.26.1.3 OFF

```
#define OFF 0
```

6.26.1.4 OK

```
#define OK 1
```

6.26.1.5 ON

```
#define ON 1
```

6.26.2 Dokumentation der benutzerdefinierten Typen

6.26.2.1 byte

```
using byte = unsigned char
```

6.26.2.2 int16

```
using int16 = short int
```

6.26.2.3 int32

```
using int32 = long int
```

6.26.2.4 int64

```
using int64 = long long int
```

6.26.2.5 int8

```
using int8 = signed char
```

6.26.2.6 uint16

```
using uint16 = unsigned short int
```

6.26.2.7 uint32

```
using uint32 = unsigned long int
```

6.26.2.8 uint64

```
using uint64 = unsigned long long int
```

6.26.2.9 uint8

```
using uint8 = unsigned char
```

6.26.2.10 word

```
using word = unsigned int
```

6.26.3 Dokumentation der Aufzählungstypen

6.26.3.1 PosState

```
enum class PosState [strong]
```

Aufzählungswerte

	Invalid
	Undefined
	Defined
	Position
	Posture
	PosAndPost

6.27 Types.hpp

[gehe zur Dokumentation dieser Datei](#)

```
00001 /*
00002  * Program: Types.hpp
00003  * Description: header file containing Global Structs
00004  * Autor: Chukwunonso Bob-Anyeji
00005  * Date: 09.06.2024
00006  */
00007
00008 // NOTE:
00009 // 1) Grosse eines Pointers im ATM-2560 Kontrollern sind 16bit
00010 // 2) Das System/Compiler basiert auf dem C++ 11 mit einem LP32 Daten Modell d.h.
00011 // 2.1) Charakter => 8 Bit
00012 // 2.2) Integer => 16 Bit
00013 // 2.3) Long Integer => 32 Bit
00014 // 2.4) Long Long Integer => 64 Bit
00015
00016 #ifndef TYPES_H_
00017 #define TYPES_H_
00018
00019 // Type Definitionen
00020 using int8 = signed char ;
00021 using uint8 = unsigned char;
00022 using byte = unsigned char;
00023
00024 using int16 = short int;
00025 using uint16 = unsigned short int;
00026 using word = unsigned int;
```

```

00027
00028 using int32 = long int;
00029 using uint32 = unsigned long int;
00030
00031 using int64 = long long int;
00032 using uint64 = unsigned long long int;
00033
00034 // Variable Definitionen
00035 #define IDENTIFIER_LEN 16
00036 #define OK 1
00037 #define NOK -1
00038 #define ON 1
00039 #define OFF 0
00040
00041 #include <WString.h>
00042
00043 /* Kinematik Struct */
00044
00045 /* Denavit Hartenberg-Parameter for all Joints */
00046 struct DhParameters{
00047     float alpha[6] = { 90.000, 0.000, 0.000, 0.000, 0.000, 0.000};
00048     float beta[6] = {-90.000, 0.000, 90.000, -90.000, 90.000, 0.000};
00049     float theta[6] = { 0.000, 0.000, 0.000, 0.000, 0.000, 0.000};
00050     float link[6] = { 0.000, 258.300, 281.516, 0.000, 74.710, 0.000};
00051     float disp[6] = { 0.000, 0.000, 0.000, 0.000, 0.000, 0.000};
00052 };
00053
00054 /* Denavit Hartenberg_Parameter Discription */
00055 struct DhParam{
00056     float alpha;
00057     float beta;
00058     float theta;
00059     float link;
00060     float disp;
00061 };
00062
00063 /* Structs Containing a 4 by 4 Matrix */
00064 struct Matrix4x4{
00065     float m11[4]; float m12[4]; float m13[4]; float m14[4];
00066 };
00067
00068 /* Position Data State */
00069 enum class PosState{
00070     Invalid = -1,
00071     Undefined = 0,
00072     Defined = 1,
00073     Position = 4,
00074     Posture = 8,
00075     PosAndPost = 16
00076 };
00077
00078 /* Location and Orientation Struct */
00079 struct Position{
00080     float X;
00081     float Y;
00082     float Z;
00083     float A;
00084     float B;
00085     float C;
00086 };
00087 /* Location and Orientation Struct */
00088 struct sPosition{
00089     short X;
00090     short Y;
00091     short Z;
00092     short A;
00093     short B;
00094     short C;
00095 };
00096
00097 /* Angular Constelation of Joints */
00098 struct Posture{
00099     float Jt1;
00100     float Jt2;
00101     float Jt3;
00102     float Jt4;
00103     float Jt5;
00104     float Jt6;
00105 };
00106 /* Angular Constelation of Joints */
00107 struct sPosture{
00108     short Jt1;
00109     short Jt2;
00110     short Jt3;
00111     short Jt4;
00112     short Jt5;
00113     short Jt6;

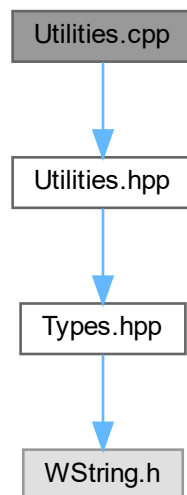
```

```
00114 };
00115
00116 /* Position and Posture Data */
00117 struct PosData{
00118     char Identifier[IDENTIFIER_LEN];
00119     sPosition Position;
00120     sPosture Posture;
00121 };
00122
00123 /* Command Struct */
00124 struct Command {
00125     int Id;
00126     String Name;
00127     int Value;
00128     String Content;
00129 };
00130
00131 #endif // TYPES_H_
```

6.28 Utilities.cpp-Dateireferenz

```
#include "Utilities.hpp"
```

Include-Abhängigkeitsdiagramm für Utilities.cpp:



Namensbereiche

- namespace `Utils`

Funktionen

- `word Utils::SetBit16` (`word` var, `int8` index)
- `word Utils::GetBit16` (`word` var, `int8` index)
- `word Utils::ResetBit16` (`word` var, `int8` index)

6.30 Utilities.hpp

[gehe zur Dokumentation dieser Datei](#)

```
00001 // =====
00002 // Datei: Utilities.hpp
00003 // Beschreibung: Header Datei, allgemeine Funktions definitionen
00004 // Autor: Chukwunonso Bob-Anyeji
00005 // Datum: 19.0812025@15-31
00006 // =====
00007
00008 #ifndef UTILITIES_H_
00009 #define UTILITIES_H_
00010
00011 #include "Types.hpp"
00012
00013 namespace Utils
00014 {
00015     word SetBit16(word var, int8 index);
00016     word GetBit16(word var, int8 index);
00017     word ResetBit16(word var, int8 index);
00018 };
00019
00020 #endif // UTILITIES_H_
```

